

PIF

Kyrylo Rudchuk

Table of Contents

Raspberry Pi	6
Install Raspberry Pi OS (64-bit).....	6
Connecting to the Raspberry Pi	6
I ² C interface	7
Static IP and Display Configuration	7
Supplied files.....	8
Docker	9
Database	10
Introduction	10
CDM	11
Relationships (CDM level)	11
LDM	12
Relationships (LDM level).....	12
SQL.....	13
Users.....	13
FriendList.....	13
Stations	14
Measurements	14
Collections	15
CollectionMeasurement	15
CollectionAccess	16
PVC mount	16
Introduction	16
Preparation	16
Machining Procedures.....	17
Assembly	18
Printed circuit	19
List of materials	19
Introduction	20

Preparation	20
Soldering.....	21
Assembling	21
Conclusion.....	22
Power supply	23
Introduction	23
Cutting the Cable.....	23
Removing the Outer Insulation.....	23
Removing the Inner Insulation Layer	24
Preparing the Wires	24
Soldering and Insulation.....	25
Reconnecting the Shielding	25
Conclusion.....	25
Website	26
Purpose of the site	26
User-facing pages (appearance, purpose, how to use)	26
Login	26
Registration.....	27
Account.....	27
AccountEdit	28
Stations	28
StationEdit.....	29
Measurements	29
MyCollections	30
Collection.....	31
EditCollection	31
Friends	32
Shared.....	32
Logout	33
Sensor #1	34
List of materials	34
Introduction	34
ADS1115 Installation.....	34

MQ135 Sensor Installation	35
Sensor Testing	35
Conclusion.....	36
Sensor #2	37
List of materials	37
Introduction	37
Sensors Installation	37
Sensors Testing.....	38
Conclusion.....	40
Web Database Server	40
Introduction	40
Commands and Configuration	41
System Update.....	41
Docker Installation	41
Verify Installation.....	42
Project Structure Creation.....	42
Docker Compose Configuration	42
Webserver Docker file.....	43
File Permissions	43
Start Docker Containers.....	44
Secure MariaDB Installation	44
Database Configuration	46
Verification	46
Website and Database Test	46
phpMyAdmin Test	46
PHP Test	47
Making sure all packages up to date.....	48
Conclusion.....	48
Assembly.....	49
List of materials	49
Introduction	49
Preparing the Opening for the Power Button	49
Assembling the Wooden Box.....	50

Mounting the Sensors	50
Connecting the Power Button	51
Conclusion.....	52
Data Transfer	52
Introduction	52
PHP Script Creation	52
Database Preparation	53
Testing Data Transfer	54
Raspberry Pi Configuration	54
Verification	55
Manual Data Entry Form	55
Conclusion.....	57
Code.....	57
Lighting.....	59
List of materials	59
Introduction	59
Preparing the Opening.....	60
Preparing the LED Strip and Wires	60
Connecting Components	61
Insulation and Mounting.....	61
Conclusion.....	61
Backup Server.....	62
Introduction	62
Web/DB Server Configuration	62
Allow External Access to MariaDB.....	62
Create Backup User.....	63
Backup Server Setup.....	64
System configuration	64
Backup Directory.....	64
Backup Script.....	65
Cron Automation	66
FTP	66
Conclusion.....	68

Code.....	68
-----------	----

Raspberry Pi

Install Raspberry Pi OS (64-bit)

1. Download the **Raspberry Pi Imager** from the official website:
<https://www.raspberrypi.com/software/>.

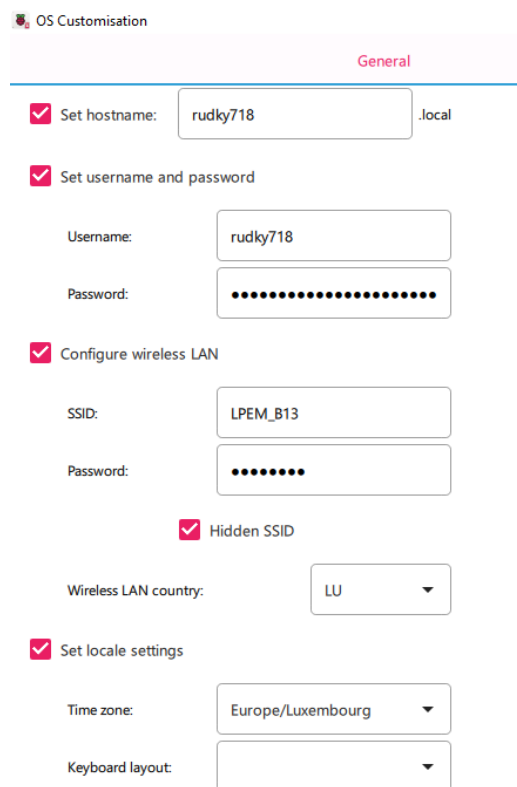
2. Insert a microSD card into your computer.

3. In Raspberry Pi Imager:

Select your **Raspberry Pi Device**

- Select **Raspberry Pi OS (64-bit)**.
- Select your microSD card.
- Click **Next**.
- Configure **Raspberry Pi OS**

4. After flashing, insert the microSD card into your Raspberry Pi and power it on.



The screenshot shows the 'OS Customisation' window with the 'General' tab selected. It contains several configuration options, each with a checked checkbox:

- Set hostname:** The text 'rudky718' is entered in the input field, followed by '.local'.
- Set username and password:** The 'Username' field contains 'rudky718'. The 'Password' field is masked with dots.
- Configure wireless LAN:** The 'SSID' field contains 'LPEM_B13'. The 'Password' field is masked with dots.
- Hidden SSID:** This checkbox is checked.
- Wireless LAN country:** A dropdown menu shows 'LU' selected.
- Set locale settings:** The 'Time zone' dropdown shows 'Europe/Luxembourg' selected. The 'Keyboard layout' dropdown is empty.

1 OS Customisation

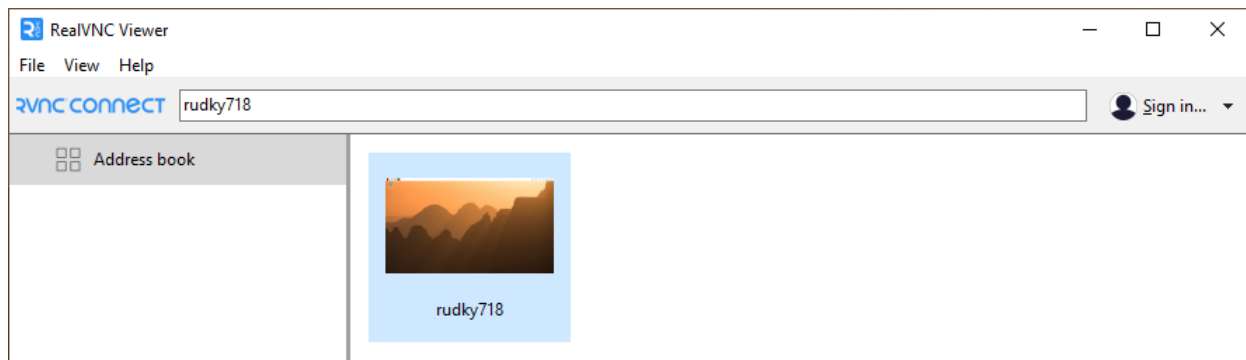
Connecting to the Raspberry Pi

There are two connection methods that I use:

- **Via PowerShell (SSH)** using the hostname configured during the OS setup.

- **Via RealVNC** for graphical interface access, also using the configured hostname.

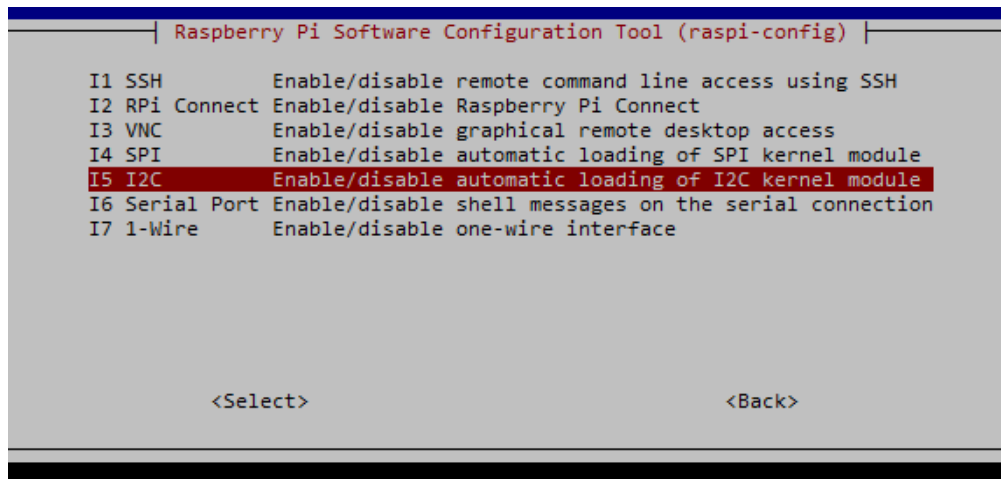
For terminal operations, PowerShell will be used. For visual interface operations, RealVNC will be used.



2 RealVNC

I²C interface

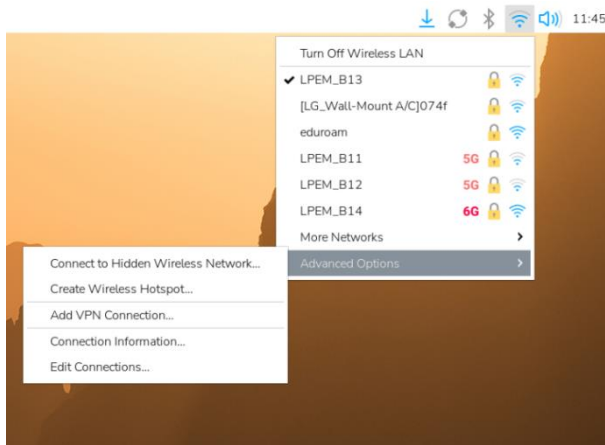
1. Open a terminal session via PowerShell and launch the Raspberry Pi configuration tool:
`sudo raspi-config`
2. Enable the **I²C interface** in interface options (required for future operations).



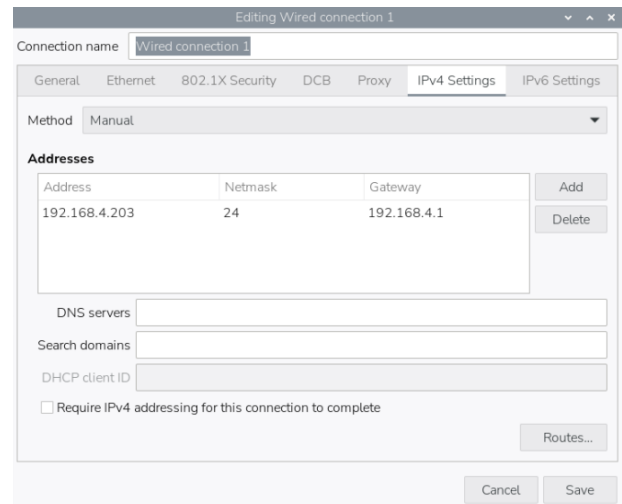
3 I2C raspi-config

Static IP and Display Configuration

1. Access the Raspberry Pi via **RealVNC** for graphical configuration.
2. Configure a **static IP address**:
 - Navigate to Network → Advanced Options → Edit Connections → IPv4.

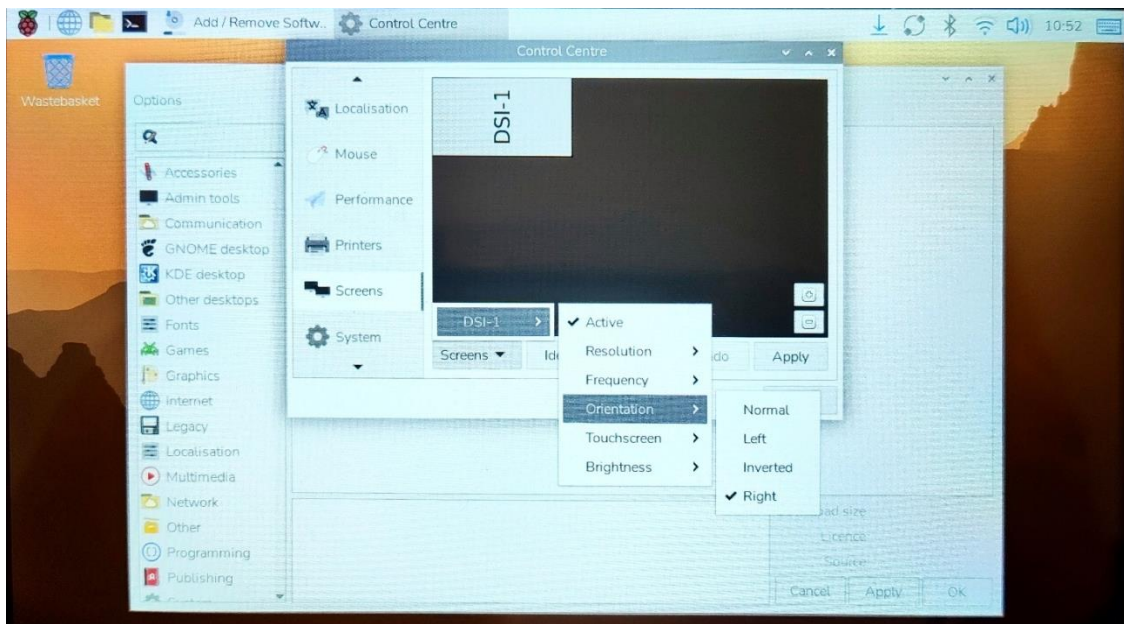


4 Network Advanced Options



5 Static IP

3. Configure the **screen orientation** in control centre as required.

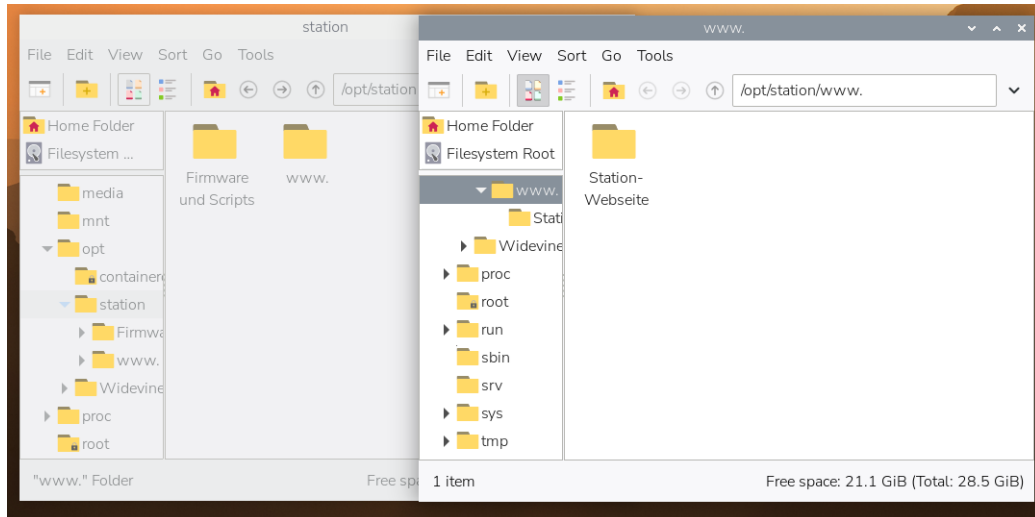


6 Screen orientation

Supplied files

1. Create the necessary directories for project files:

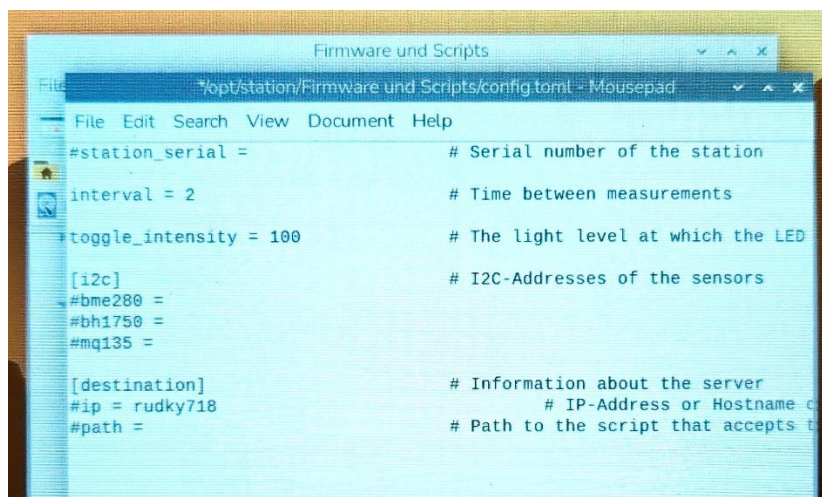

```
sudo mkdir -p /opt/station/www
sudo mkdir -p /opt/station
sudo chmod -R 755 /opt/station
```
2. Transfer the supplied files from a USB stick to the Raspberry Pi:
 - **Website files** → /opt/station/www.
 - **Firmware files, configuration file, and sensor testing scripts** → /opt/station



7 Folders

3. Configuration File Adjustment

Open the configuration file located among the transferred files
/opt/station/config.toml



8 config.toml

Docker

1. Download and install Docker using the official script:

```
curl -fsSL https://get.docker.com -o get-docker.sh
sudo sh get-docker.sh
```
2. Launch a Docker container:

```
docker run -d --name php --restart unless-stopped -p 8080:80 -v /tmp/station:/tmp/station -v /opt/station/www:/var/www/html php:8.4-apache
```

```
rudky718@rudky718: ~
File Edit Tabs Help

rudky718@rudky718:~ $ sudo docker run -d --name php --restart unless-stopped -p
8080:80 -v /tmp/station:/tmp/station -v /opt/station/www:/var/www/html php:8.4-a
pache
Unable to find image 'php:8.4-apache' locally
8.4-apache: Pulling from library/php
b89cf3ec7a3e: Pull complete
65dd1b4d69f9: Pull complete
b6a214c39dd4: Pull complete
22003554cbf4: Pull complete
4f4fb700ef54: Pull complete
721a3cb0d966: Pull complete
12ee6a10c3fa: Pull complete
8caf2c340c63: Pull complete
7e0e8bc13627: Pull complete
c854e8ba3592: Pull complete
31a82d23f7fc: Pull complete
def40a9103a7: Pull complete
53138ea265d8: Pull complete
3fa20110e5dd: Pull complete
d71c20bd3449: Pull complete
Digest: sha256:2e8ef5f8a353d88fd93fa8b1ec6549511701bed888c6d2cb20d5c774f9dae4ee
Status: Downloaded newer image for php:8.4-apache
dcd909762f6044679187e61aaa876c4882fd9c1507447b4aa5e36ad81f26792b
rudky718@rudky718:~ $
```

9 Docker container launch

3. Configure Docker to run without sudo:
sudo groupadd docker
sudo usermod -aG docker \$USER
4. Verify that the container is running and that Docker can be used without sudo:
docker ps

```
rudky718@rudky718:~ $ docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED    STATUS    PORTS
dcd909762f60   php:8.4-a "docker-php-entrypoi..." 3 hours ago Up About a minute 0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
```

10 docker ps

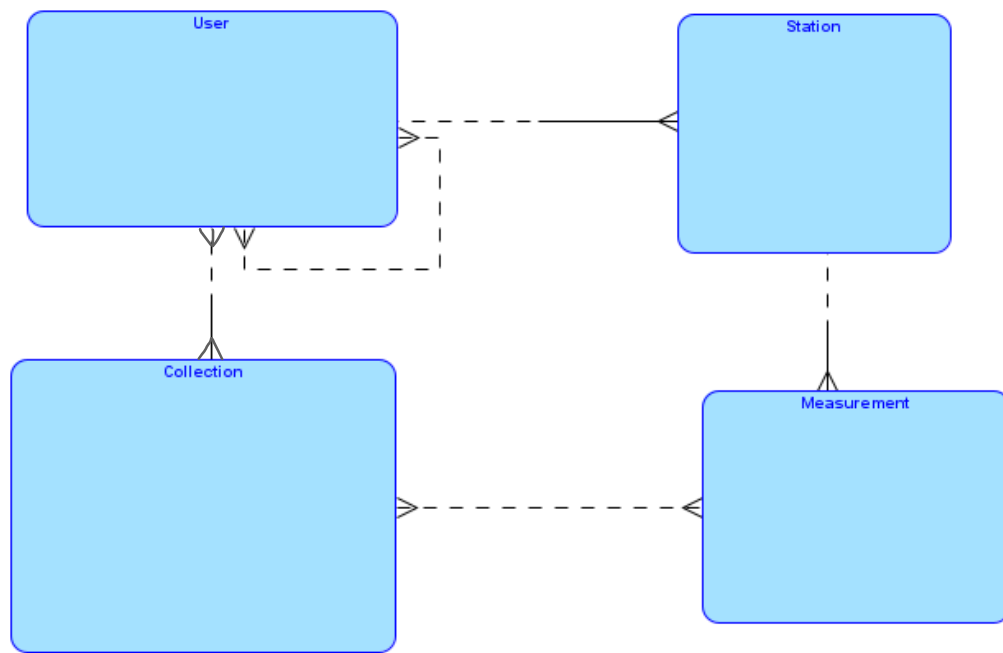
Database

Introduction

This document describes the database design and implementation for the measurement-station project. The system stores environmental measurements (temperature, humidity, air pressure, light intensity, air quality) produced by devices called stations. Users (customers and administrators) sign up, register stations, view measurement data, build collections of measurements and share collections with friends (view-only). The goal of the database is to reliably store and expose data for the website, enforce essential integrity, and be easy to query

for UI features such as listing stations, filtering measurements by time range and sharing collections.

CDM



11 CDM

Relationships (CDM level)

User 1 : N Station

Station 1 : N Measurement

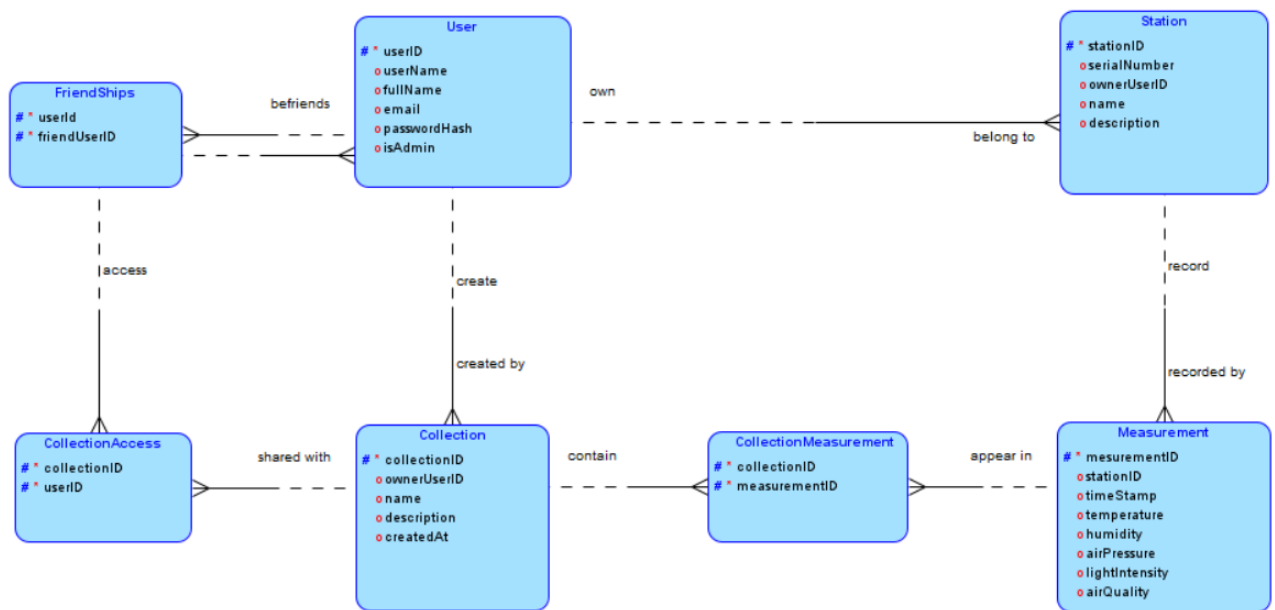
User 1 : N Collection

Collection M : N Measurement

User M : N User (self many-to-many)

Collection M : N User

LDM



12 LDM

Relationships (LDM level)

User can own one or many **stations**.

Station must belong to one and only one **user**.

Station can produce one or many **measurements**.

Measurement must be recorded by one and only one **station**.

User can create one or many **collections**.

Collection must be created by one and only one **user**.

Collection can contain many **measurements**.

Measurement can appear in many **collections**.

User can befriend many **users**.

Friendship always links two and only two **users**.

Collection can be shared with many **users**.

User can access many **collections** shared with him.

SQL

Users

```
userID INT AUTO_INCREMENT PRIMARY KEY,  
username VARCHAR(50) NOT NULL UNIQUE,  
fullName VARCHAR(100) NOT NULL,  
email VARCHAR(100) NOT NULL UNIQUE,  
passwordHash VARCHAR(255) NOT NULL,  
isAdmin BOOLEAN NOT NULL DEFAULT FALSE
```

- **Purpose:** Central entity of the system. Stores all registered accounts, including both normal users and administrators. Users are the foundation for ownership of stations, creation of collections, friendships, and access rights. Without users, no other entity can exist in the system.
- **Primary Key:** userID
- **Foreign Keys:** None directly, but referenced by other tables.
- **Usage in other tables:**
 - FriendList (friendship relations)
 - Stations (station ownership)
 - Collections (collection creator)
 - CollectionAccess (shared access to collections)

FriendList

```
userID1 INT NOT NULL,  
userID2 INT NOT NULL,  
PRIMARY KEY(userID1, userID2),  
FOREIGN KEY(userID1) REFERENCES Users(userID) ON DELETE CASCADE,  
FOREIGN KEY(userID2) REFERENCES Users(userID) ON DELETE CASCADE,  
CHECK (userID1 <> userID2)
```

- **Purpose:** Implements friendships between users. This table enables social interaction and sharing features by linking two distinct users together. It is essential for enabling collection sharing, since only friends can be granted access.
- **Primary Key:** (userID1, userID2) (composite)
- **Foreign Keys:**
 - userID1 → Users.userID
 - userID2 → Users.userID
- **Usage in other tables:**

- Indirectly supports CollectionAccess by defining who can be shared with.

Stations

```
stationID INT AUTO_INCREMENT PRIMARY KEY,  
serialNumber VARCHAR(20) NOT NULL UNIQUE,  
name VARCHAR(100) NOT NULL,  
description TEXT,  
ownerUserID INT NOT NULL,  
FOREIGN KEY(ownerUserID) REFERENCES Users(userID) ON DELETE CASCADE
```

- **Purpose:** Represents physical measurement devices owned by users. Each station has a unique company-assigned serial number and can be named and described by the customer. Stations are the source of all measurement data in the system.
- **Primary Key:** stationID
- **Foreign Keys:**
 - ownerUserID → Users.userID
- **Usage in other tables:**
 - Measurements (records sensor data linked to stations)

Measurements

```
measurementID INT AUTO_INCREMENT PRIMARY KEY,  
stationID INT NOT NULL,  
timestamp DATETIME NOT NULL,  
temperature FLOAT NOT NULL,  
humidity FLOAT NOT NULL,  
airPressure FLOAT NOT NULL,  
lightIntensity FLOAT NOT NULL,  
airQuality FLOAT NOT NULL,  
FOREIGN KEY(stationID) REFERENCES Stations(stationID) ON DELETE CASCADE
```

- **Purpose:** Stores environmental readings collected from stations. Each measurement is tied to one station and contains timestamped values such as temperature, humidity, air pressure, light intensity, and air quality. This table is the core data source for analysis and visualization.
- **Primary Key:** measurementID
- **Foreign Keys:**
 - stationID → Stations.stationID
- **Usage in other tables:**
 - CollectionMeasurement (groups measurements into collections)

Collections

```
collectionID INT AUTO_INCREMENT PRIMARY KEY,  
creatorUserID INT NOT NULL,  
name VARCHAR(100) NOT NULL,  
description TEXT,  
FOREIGN KEY(creatorUserID) REFERENCES Users(userID) ON DELETE CASCADE
```

- **Purpose:** Logical grouping of measurements created by users. Collections allow customers to organize data (e.g., by time period, station, or analysis purpose). They can be empty or filled with measurements. Collections are also the unit of sharing between friends.
- **Primary Key:** collectionID
- **Foreign Keys:**
 - creatorUserID → Users.userID
- **Usage in other tables:**
 - CollectionMeasurement (links collections to measurements)
 - CollectionAccess (grants access to other users)

CollectionMeasurement

```
collectionID INT NOT NULL,  
measurementID INT NOT NULL,  
PRIMARY KEY(collectionID, measurementID),  
FOREIGN KEY(collectionID) REFERENCES Collections(collectionID) ON DELETE  
CASCADE,  
FOREIGN KEY(measurementID) REFERENCES Measurements(measurementID) ON DELETE  
CASCADE
```

- **Purpose:** Junction table that enables many-to-many relationships between collections and measurements. It allows one measurement to be part of multiple collections and one collection to contain multiple measurements.
- **Primary Key:** (collectionID, measurementID) (composite)
- **Foreign Keys:**
 - collectionID → Collections.collectionID
 - measurementID → Measurements.measurementID
- **Usage in other tables:**
 - Acts as a bridge between Collections and Measurements.

CollectionAccess

```
collectionID INT NOT NULL,  
userID INT NOT NULL,  
PRIMARY KEY(collectionID, userID),  
FOREIGN KEY(collectionID) REFERENCES Collections(collectionID) ON DELETE  
CASCADE,  
FOREIGN KEY(userID) REFERENCES Users(userID) ON DELETE CASCADE
```

- **Purpose:** Manages sharing of collections with other users. This table defines which users have read-only access to a given collection. It enables collaboration and data sharing while preserving ownership rights for the creator.
- **Primary Key:** (collectionID, userID) (composite)
- **Foreign Keys:**
 - collectionID → Collections.collectionID
 - userID → Users.userID
- **Usage in other tables:**
 - Provides access control for collections, based on friendships defined in FriendList.

PVC mount

Introduction

This report describes the machining and assembly steps required to prepare the PVC base plate and PVC display mount, followed by the attachment of the Raspberry Pi and touch display. All actions had to be performed carefully and confidently, as no spare parts were available.

Preparation

Before beginning any machining work, each PVC part had to be prepared. All components (PVC base plate and PVC display mount) had to be marked with the personal IAM code using a permanent marker to ensure traceability. Precise reference markings for drilling had to be made using a fine scribing tool (scraper) to achieve accurate guidelines.

Additionally, at each marked drilling point, a small indentation had to be made to allow the drill bit to self-center more easily during drilling and prevent slipping.

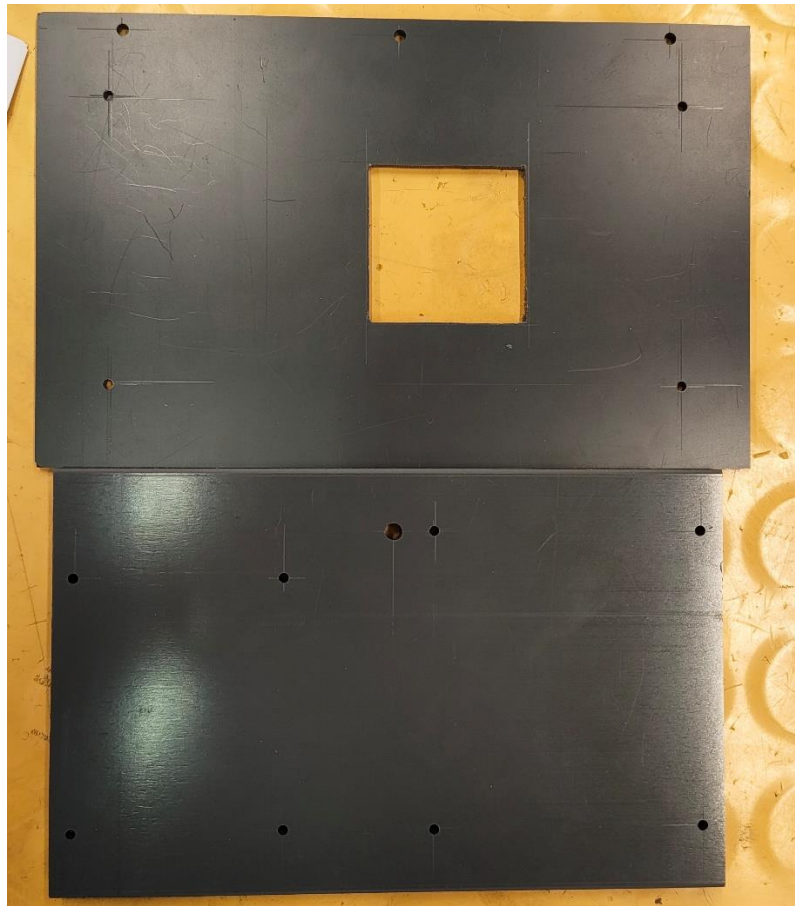
These markings and indentations had to be checked against the dimensional documentation. Finally, all positions had to be verified by comparing them with the teacher-approved model to ensure proper alignment and avoid potential errors before machining began.



13 Scraped PVC

Machining Procedures

After the marked and indented drill points were verified, drilling had to be carried out using the appropriate drill bit diameters (2.3 mm, 2.5 mm, 3 mm, and 4 mm depending on the function of each hole). After drilling, all holes had to be deburred. The opening for the display cable had to be made by drilling four corner holes, then cutting the interior with a saw, and finally smoothing and finishing the edges using a sanding stick.



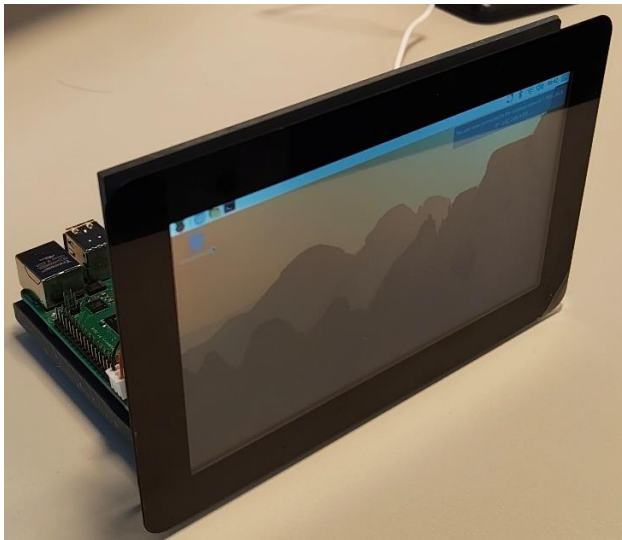
14 Drilled PVC

Assembly

The PVC display mount had to be fixed to the base plate. The cables had to be disconnected from the Raspberry Pi, routed through the machined opening, and reconnected to the display after assembly.

The touch display had to be mounted to the PVC display mount using the supplied 5 mm spacer sleeves. The Raspberry Pi had to be attached to the base plate using 5 mm spacer sleeves.

Once all components were aligned and secured, the assembly was complete.



15 Assemble result from the angle



16 Assemble result from the side

Printed circuit

List of materials

Component	Quantity
2-layer printed circuit board 75 x 85 mm	1
40-pin straight connector strip R WS40G	1
GPIO ribbon cable for Raspberry Pi Model B+ (40 pins)	1
Metal resistor 0207 0.6W 10K	1
Metal resistor 0207 0.6W 56K	1
BC517 transistor	1

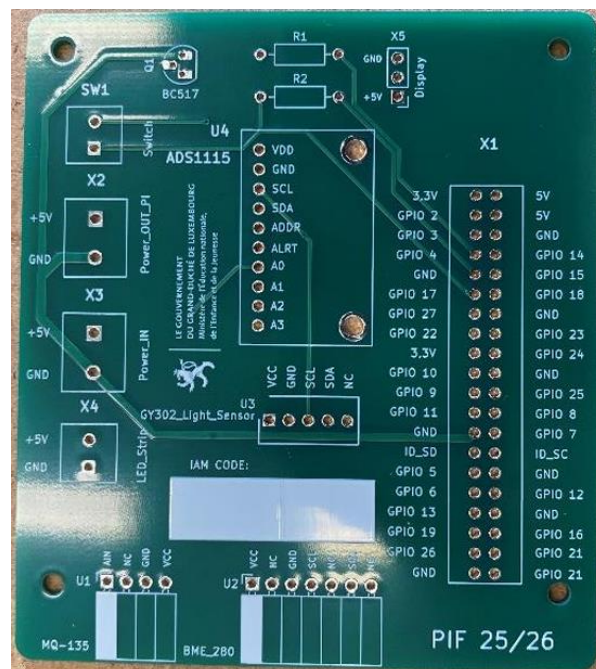
PCB connection terminal 2-pin RM 5	2
PCB terminal block, 2-pin RM 3.5	2
1-row angled socket strip 1 x 4 -RM2.54mm h=8mm	1
1-row angled female connector strip 1 x 7 -RM2.54mm h=8mm	1
Pin strip, 1 row, straight, 1 x 3 - RM2.54 mm	1
PCB connector, straight pin header with female connector (set with cable)	1
M3 x 12 countersunk head	3
M2.5 x 20 countersunk head	8
ADS1115 ADC module I2C (Extra)	1

Introduction

The objective of this task is to assemble a functional Printed Circuit Board (PCB) by soldering electronic components onto it. The PCB provides both mechanical support and electrical connections, allowing integration with other sensors. Proper soldering is essential to avoid short circuits and unreliable connections.

Preparation

Before soldering, the documentation must be reviewed to reduce the risk of faults. Also, use a digital multimeter to check the resistors and verify that their actual resistance values match the required specifications before installing them on the board.



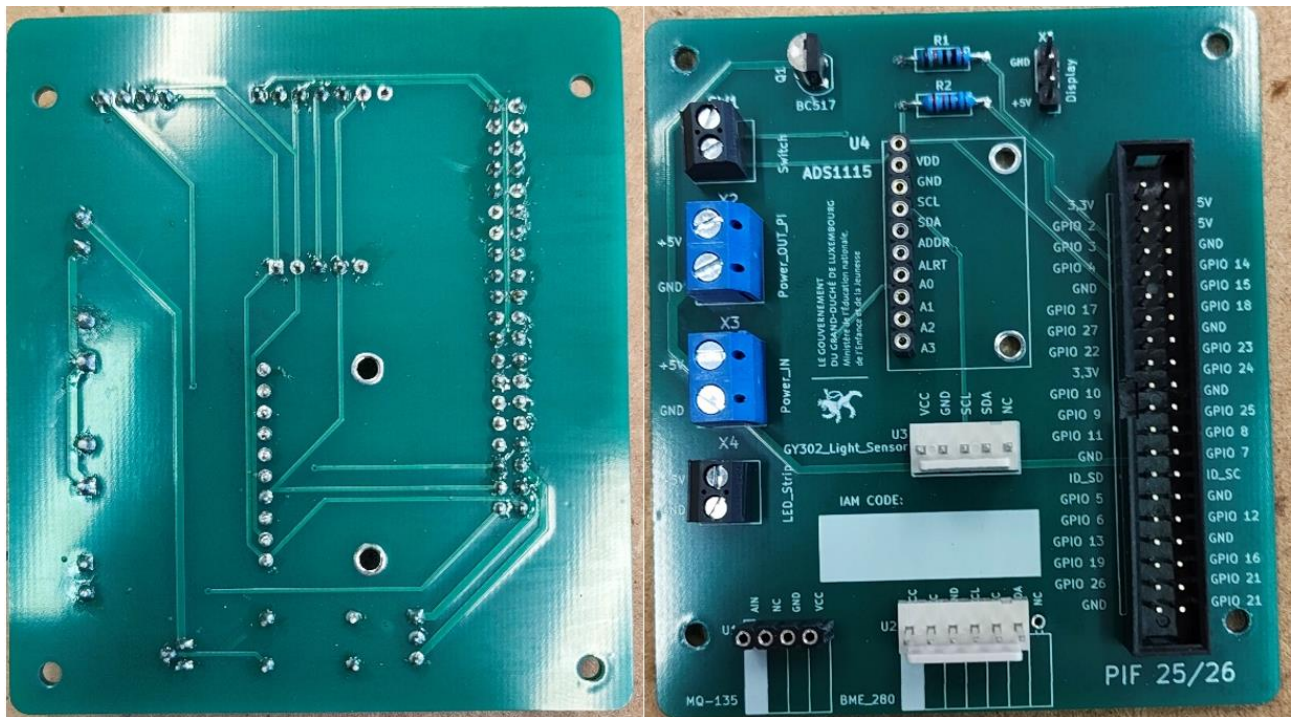
17 PCB

Soldering

Components must be placed in their designated footprints and soldered correctly.

Guidelines:

- Keep the soldering area clean and free of dust.
- Use an appropriate amount of solder.
- Solder joints should be smooth and shiny.
- Avoid overheating the PCB to prevent damage.
- After soldering, electrical connections could be checked with a multimeter.
- Any protruding component leads or pins must be trimmed after soldering.



18 Soldered PCB

Assembling

Mounting:

- After soldering, attach the PCB to the plastic (PVC) panel using screws.
- Do not overtighten the screws to avoid damaging the board.

Cable connection:

- Connect the cables according to the reference or documentation provided.
- Handle components with weak pins carefully and avoid applying excessive force.

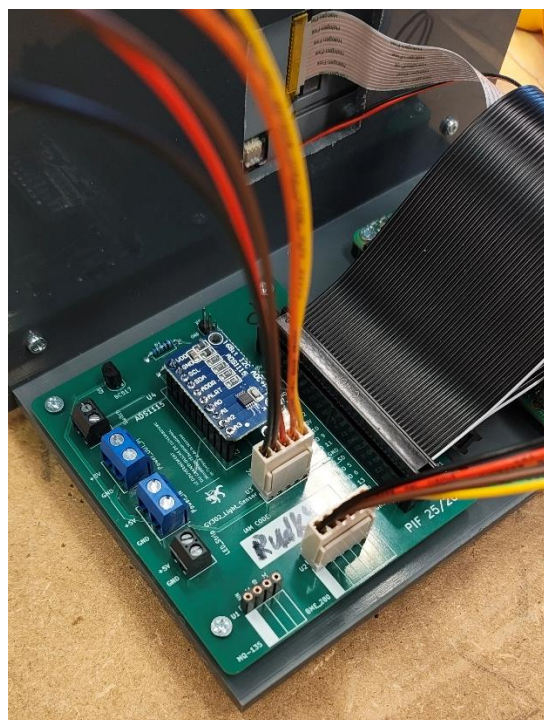


19 Assembled PSB

Conclusion

The PCB has been successfully assembled. All components are securely mounted and properly soldered. The board is ready for further sensor integration and future development stages.

As an addition, an extra module was soldered onto the PCB. This component is intended for future sensor integration.



20 Assembled PCB from another angle

Power supply

Introduction

The goal of this work is to learn how to repair a damaged cable using wire stripping and soldering. A multi-layer cable consists of an outer insulation, metallic shielding, an inner protective layer, and four internal wires. By carefully cutting, stripping, and soldering these parts, the cable can be safely restored.



Cutting the Cable

Initially, the cable is undamaged.

The first action is to cut the cable in half using wire cutters. This creates two separate cable ends that will later be reconnected.



Removing the Outer Insulation

Next, the outer insulation layer is removed.

The outer layer must be cut very carefully, because directly underneath it there is a metallic braided shielding. This shielding is easy to damage and must remain intact.

After exposing the metallic braid, it is carefully folded back onto the intact part of the cable so that it does not interfere with further work.



Removing the Inner Insulation Layer

Under the shielding, the inner protective layer is removed, exposing the four internal wires. At this stage, damaging the wires is unlikely, but care is still required.



Preparing the Wires

The internal wires are aligned and cut at different lengths so that the solder joints are not located in the same position. This staggered method reduces the risk of short circuits.

A small amount of insulation is removed from each wire to expose the copper conductor.



Soldering and Insulation

Before soldering, heat-shrink tubing is placed on the wires. After soldering, it will no longer be possible to add it.

The exposed copper ends are soaked with solder, then soldered together. Each joint is held steady until the solder solidifies.

After soldering, the heat-shrink tubing is moved over the joints and heated until it shrinks tightly, providing proper insulation.

The wires must remain straight, without angles or protruding parts.

Reconnecting the Shielding

The metallic braided shielding is twisted into a single strand and reconnected using the same soldering method.



Conclusion

In conclusion, this work demonstrates how a multi-layer cable can be safely repaired. By carefully handling each layer, correctly soldering the internal wires, and restoring the metallic shielding, the cable regains its electrical functionality.

Website

Purpose of the site

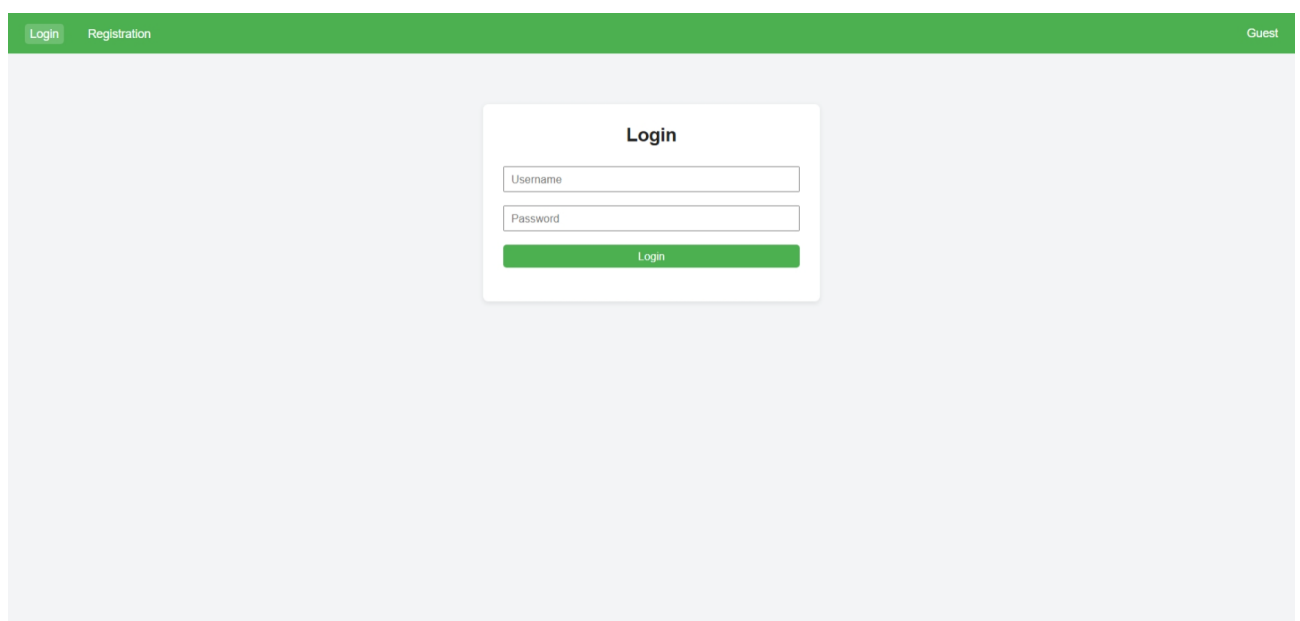
This site is built to manage stations and their measurements. The main user capabilities are:

- register and log in;
- claim existing stations by serial number and edit them;
- view measurements (temperature, humidity, pressure, light intensity, air quality) per station and for a chosen time range;
- create collections (groups) of measurements from the visible table and manage your collections;
- manage friends and share collections (the UI for sharing exist; server-side requires further implementation).

User-facing pages (appearance, purpose, how to use)

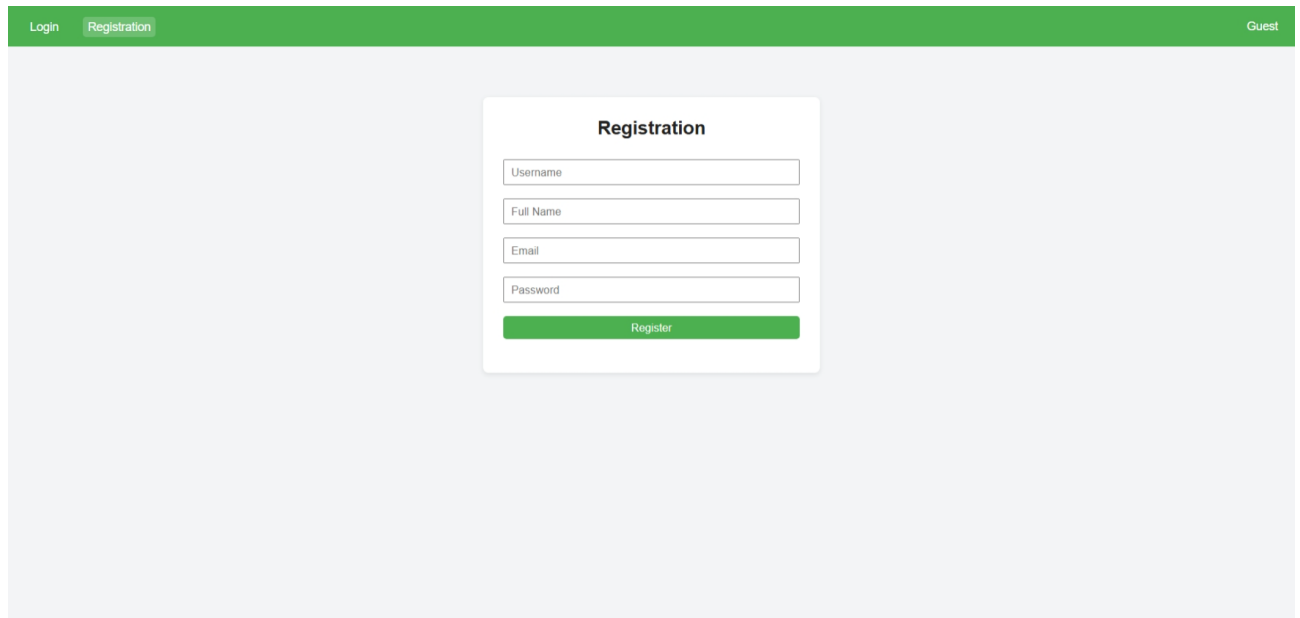
Login

- Appearance: a centered card with Username and Password fields and a "Login" button.
- Purpose: authenticate existing users.
- How to use: enter username and password; on success the user is redirected to Account.php and a session is established.

A screenshot of a web application's login page. At the top, there is a green navigation bar with the text 'Login' and 'Registration' on the left, and 'Guest' on the right. The main content area has a light gray background. In the center, there is a white card with a green border. The card has the title 'Login' at the top. Below the title, there are two input fields: 'Username' and 'Password'. At the bottom of the card, there is a green button with the text 'Login'.

Registration

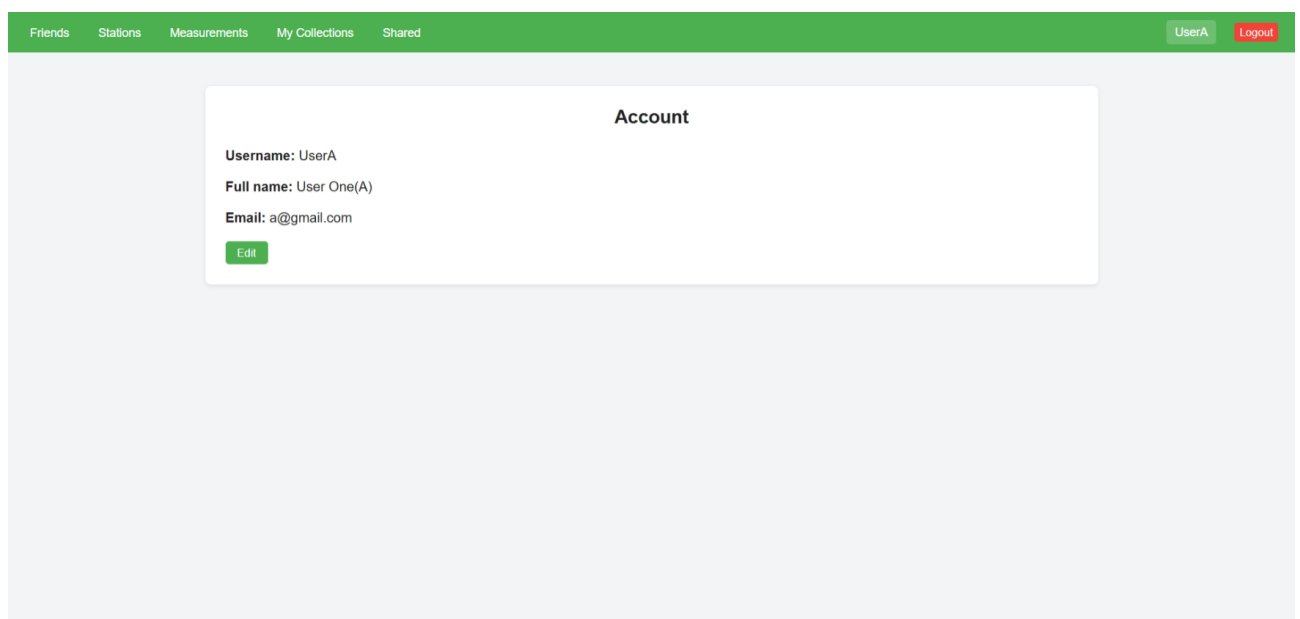
- Appearance: a registration card with fields Username, Full Name, Email, Password.
- Purpose: create a new user account.
- How to use: fill in required fields and press "Register". The app validates uniqueness of username and email and stores a hashed password.



The screenshot shows a web application interface with a green header bar. On the left of the header are links for 'Login' and 'Registration' (the latter is highlighted). On the right is a 'Guest' link. The main content area is light gray and contains a white 'Registration' card. The card has a title 'Registration' and four input fields: 'Username', 'Full Name', 'Email', and 'Password'. Below these fields is a green 'Register' button.

Account

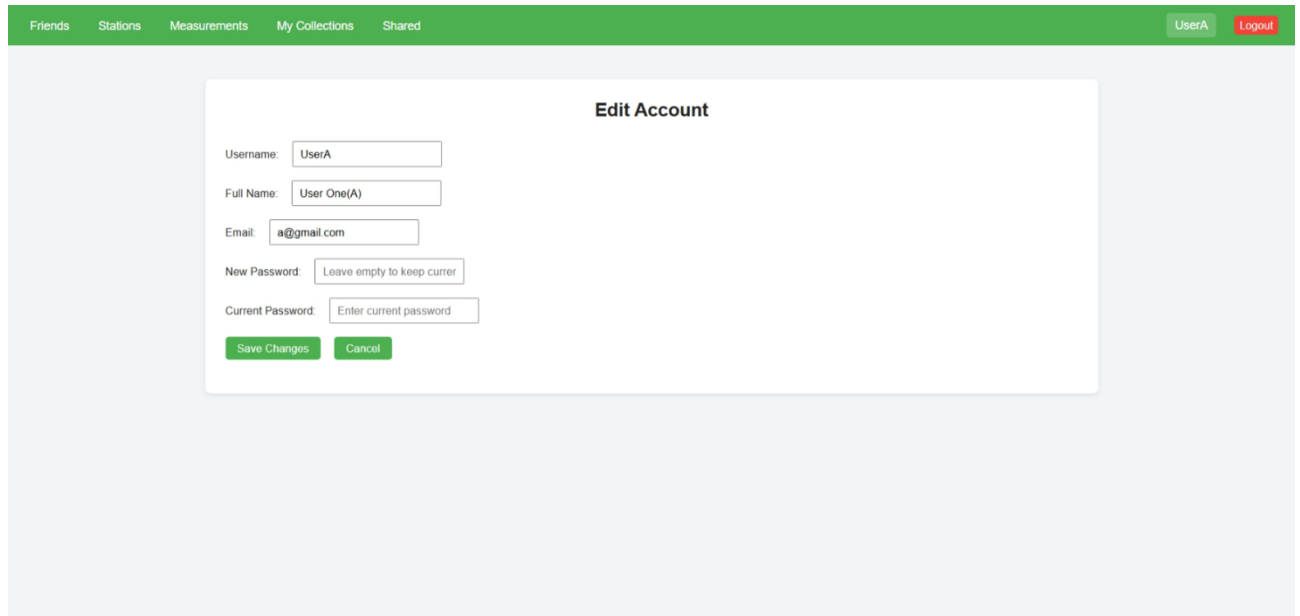
- Appearance: a card showing Username, Full name, Email and an "Edit" button.
- Purpose: provide a quick overview of the currently logged-in user's profile.
- How to use: view your data and press "Edit" to open AccountEdit.php if you need to change profile details.



The screenshot shows a web application interface with a green header bar. On the left are links for 'Friends', 'Stations', 'Measurements', 'My Collections', and 'Shared'. On the right are links for 'UserA' and 'Logout'. The main content area is light gray and contains a white 'Account' card. The card has a title 'Account' and displays the following information: 'Username: UserA', 'Full name: User One(A)', and 'Email: a@gmail.com'. Below this information is a green 'Edit' button.

AccountEdit

- Appearance: a form with fields to edit Username, Full Name, Email, New Password and Current Password (required for confirmation).
- Purpose: update user profile and optionally change the password.
- How to use: enter updated values and your current password to confirm. On success the database and session values are updated.



The screenshot shows a web application interface with a green header bar containing navigation links: Friends, Stations, Measurements, My Collections, and Shared. On the right side of the header, there is a 'UserA' button and a red 'Logout' button. The main content area is light gray and features a white 'Edit Account' form. The form has the following fields: 'Username' with the value 'UserA', 'Full Name' with the value 'User One(A)', 'Email' with the value 'a@gmail.com', 'New Password' with a placeholder 'Leave empty to keep currer', and 'Current Password' with a placeholder 'Enter current password'. At the bottom of the form are two green buttons: 'Save Changes' and 'Cancel'.

Stations

- Appearance: a form to claim a station (fields: Serial number, Station name, Description) and a list of the user's claimed stations with Edit/Delete buttons.
- Purpose: manage stations you own.
- How to use: to claim a station, enter its serial number and a display name, then press "Add station" — the station will be assigned to you only if it exists in the database and has no owner. Use Delete to release ownership.

Stations

Serial number: Station name:

Description:

Station 1 (SN-1001) Description for the Station SN-1001	Edit Delete
Station 2 (SN-1011) Description for the Station SN-1001	Edit Delete

StationEdit

- Appearance: a form to edit Station name and Description; the serial number is shown read-only.
- Purpose: edit station metadata if you are the owner.
- How to use: open via Edit from Stations.php or by supplying sn in the URL, change fields and save.

Edit Station

Serial number:

Station name:

Description:

Measurements

- Appearance: a station selector (drop-down of your stations), Start and End datetime-local inputs, a "Show" button and a measurements table. A "Create collection" button opens a modal for naming a new collection.
- Purpose: view station measurements and create a collection from the currently visible measurements.

- How to use: select a station and optionally a date range, press "Show"; the table will populate with matching measurements. Click "Create collection" to open the modal, provide a name and optional description — the server will then create a collection containing all measurements that match the current filters.

Station Measurements

Station: SN-1001 – Station 1

Start: 15.01.2026 04:13 End: 18.01.2026 22:09 Show Create collection

Timestamp	Temperature (°C)	Humidity (%)	Pressure (hPa)	Light (lux)	Gas (ppm)
2026-01-15 12:00	23.9	43	1012.4	820	15.4
2026-01-15 11:00	23.5	44	1012.3	700	15.1
2026-01-15 10:00	22.9	45.5	1012.2	550	14.7
2026-01-15 09:00	22	47	1012	350	14
2026-01-15 08:00	21	49	1011.8	180	13.2
2026-01-15 07:00	20.1	51	1011.5	70	12.5
2026-01-15 06:00	19.3	53	1011.2	15	12
2026-01-15 05:00	18.7	55	1011	2	11.7

Station Measurements

Station: SN-1001 – Station 1

Start: 15.01.2026 04:13 End: 18.01.2026 22:09 Show Create collection

Create collection

Name

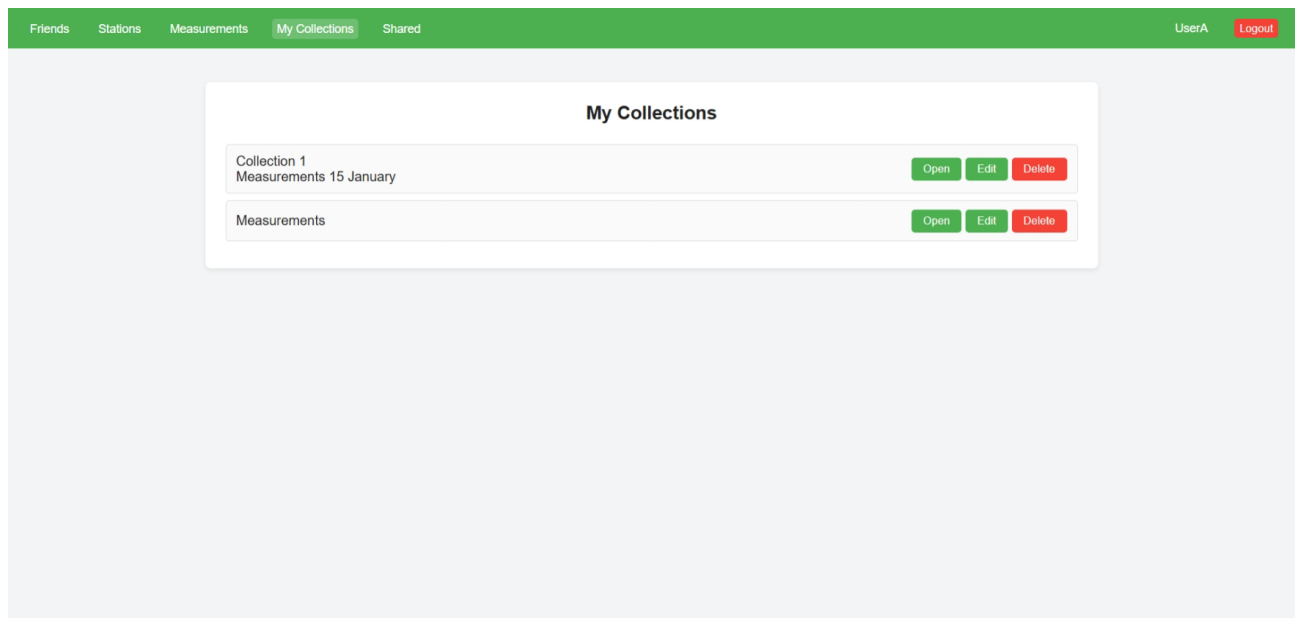
Description

Cancel Save

Timestamp	Temperature (°C)	Humidity (%)	Pressure (hPa)	Light (lux)	Gas (ppm)
2026-01-15 12:00	23.9	43	1012.4	820	15.4
2026-01-15 11:00	23.5	44	1012.3	700	15.1
2026-01-15 10:00	22.9	45.5	1012.2	550	14.7
2026-01-15 09:00	22	47	1012	350	14
2026-01-15 08:00	21	49	1011.8	180	13.2
2026-01-15 07:00	20.1	51	1011.5	70	12.5
2026-01-15 06:00	19.3	53	1011.2	15	12
2026-01-15 05:00	18.7	55	1011	2	11.7

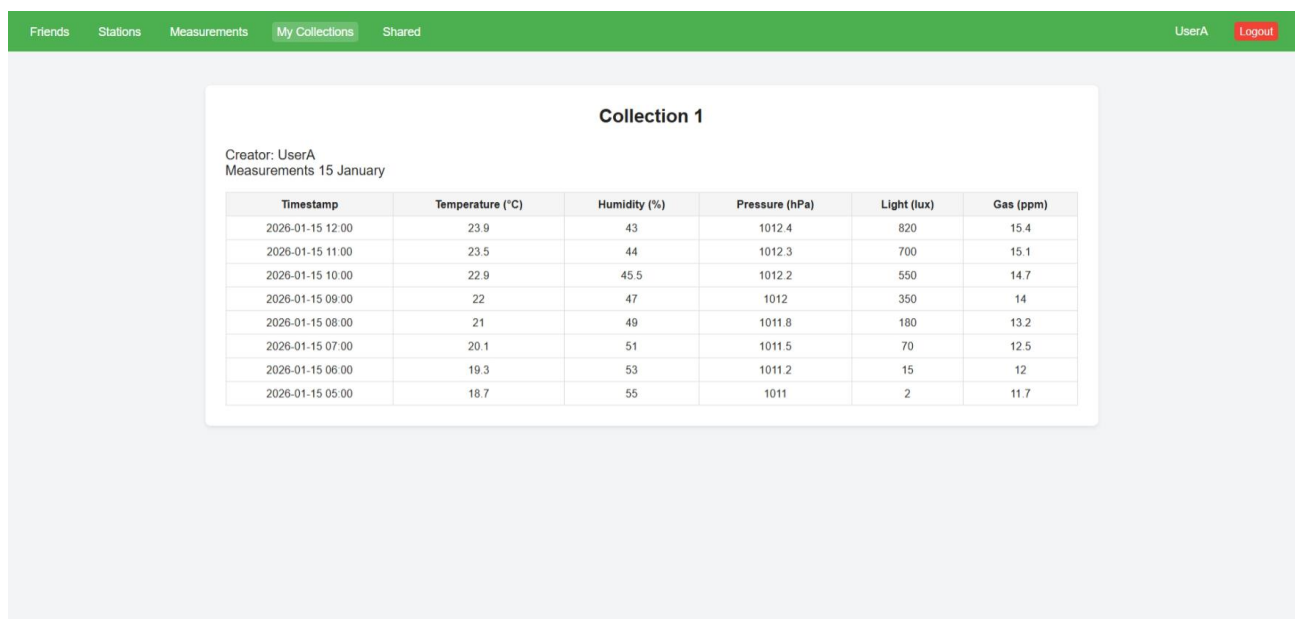
MyCollections

- Appearance: a list of your collections where each item includes name, description and buttons Open, Edit, Delete.
- Purpose: manage collections you created.
- How to use: open a collection to view it, edit metadata via Edit, or remove it with Delete (confirmation required).



Collection

- Appearance: a card showing collection name, the collection owner, and a table listing the measurements that belong to the collection.
- Purpose: inspect the contents of a single collection.
- How to use: open the page using the Open button in MyCollections.php or via `Collection.php?cid=<ID>` to view measurement entries and collection metadata.



EditCollection

- Appearance: a form to edit collection name and description.
- Purpose: allow the collection owner to update collection metadata.
- How to use: open the page using the Edit button in MyCollections.php or via `EditCollection.php?cid=<ID>`, modify fields, and press "Save Changes".

The screenshot shows a web application interface with a green header bar containing navigation links: 'Friends', 'Stations', 'Measurements', 'My Collections' (which is highlighted), and 'Shared'. On the right side of the header, it says 'UserA' and has a red 'Logout' button. The main content area is light gray and contains a white modal box titled 'Edit Collection'. Inside the modal, there are two input fields: 'Name' with the value 'Collection 1' and 'Description' with the value 'Measurements 15 January'. Below these fields are two green buttons: 'Save Changes' and 'Cancel'.

Friends

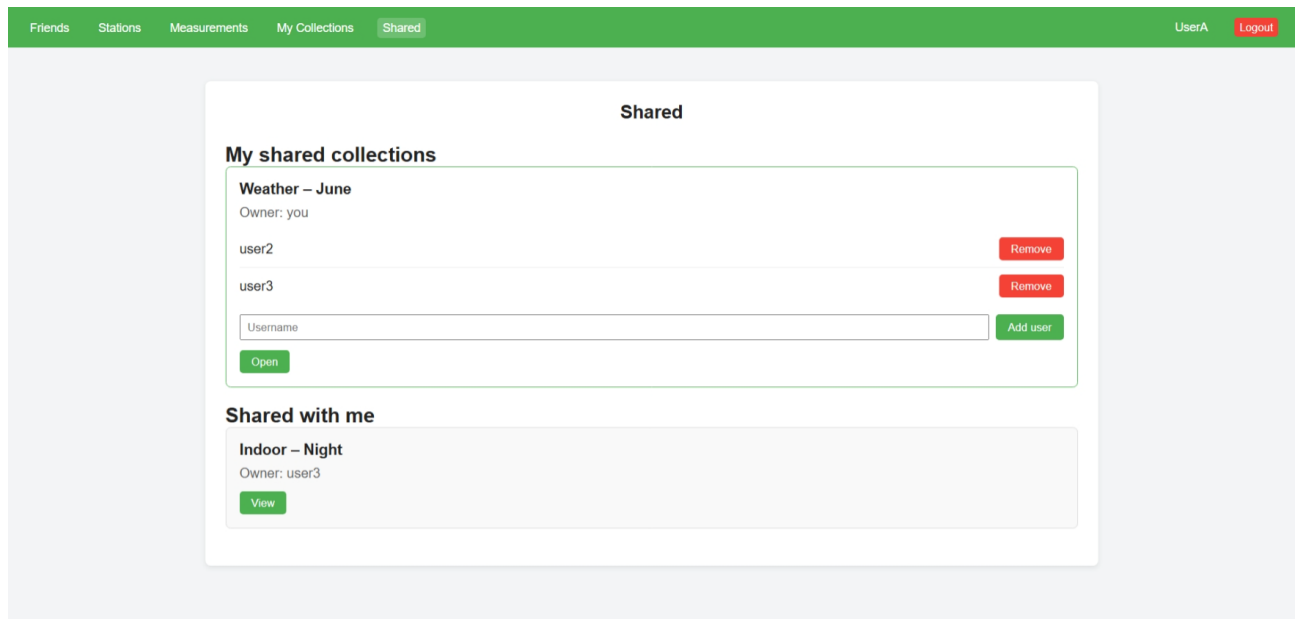
- Appearance: an input to enter a friend's username and an "Add friend" button; below is the list of friends with individual "Delete" buttons.
- Purpose: maintain a simple friend list to connect users.
- How to use: type a username and click "Add friend"; to remove a friend click "Delete" next to the desired username.

The screenshot shows the same web application interface as before, but the 'My Collections' link is not highlighted. The 'Friends' link in the header is highlighted. The main content area is light gray and contains a white modal box titled 'Friends'. Inside the modal, there is an input field labeled 'Friend username' and a green 'Add friend' button. Below this, there is a list of friends. The first friend is 'UserB', which is displayed in a light gray box with a red 'Delete' button to its right.

Shared

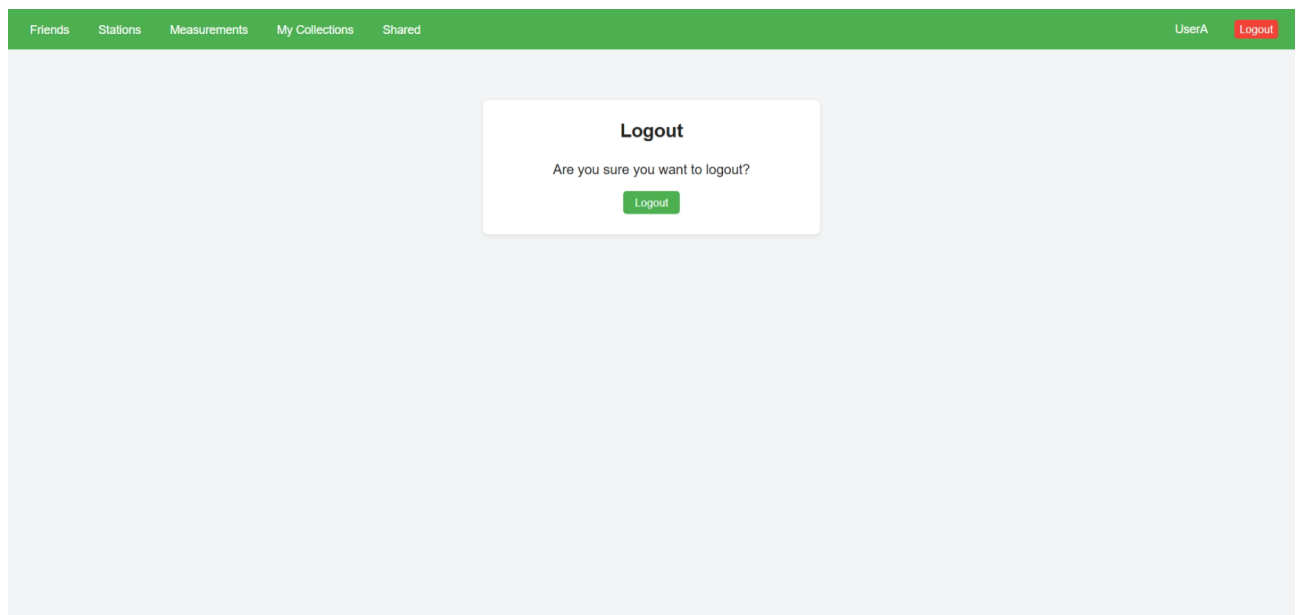
- Appearance: sections titled "My shared collections" and "Shared with me" with layout for collection cards and controls to show collection, add or remove shared users.
- Purpose: manage who has access to your collections and view collections shared with you. Current file provides only UI.

- How to use: use the add/remove controls to manage sharing, open a collection to view it.



Logout

- Appearance: a simple confirmation card asking, "Are you sure you want to logout?" and a "Logout" button.
- Purpose: end the user's session.
- How to use: click "Logout" to destroy the session and return to the login page.



Sensor #1

List of materials

Component	Quantity
ADS1115 ADC module I2C	1
MQ135 gas sensor	1

Introduction

The aim of this assignment was to connect and test an MQ135 gas sensor to measure air quality.

The sensor is used together with an ADS1115 16-bit converter, which allows the Raspberry Pi to read analog values.

The ADS1115 module was installed earlier during Userstory A2 – PCB and is used in this assignment.

This task focuses on connecting the gas sensor and verifying that the full system works correctly.

ADS1115 Installation

During Userstory A2 – PCB, the ADS1115 converter module was added to the custom printed circuit board as an extra component for future sensor integration.

The ADS1115 module was placed in its correct position on the PCB and soldered carefully using a soldering iron and solder.

The solder joints were checked visually to ensure good electrical contact and mechanical stability.

This installation made it possible to connect analog sensors, such as the MQ135 gas sensor, in later assignments.



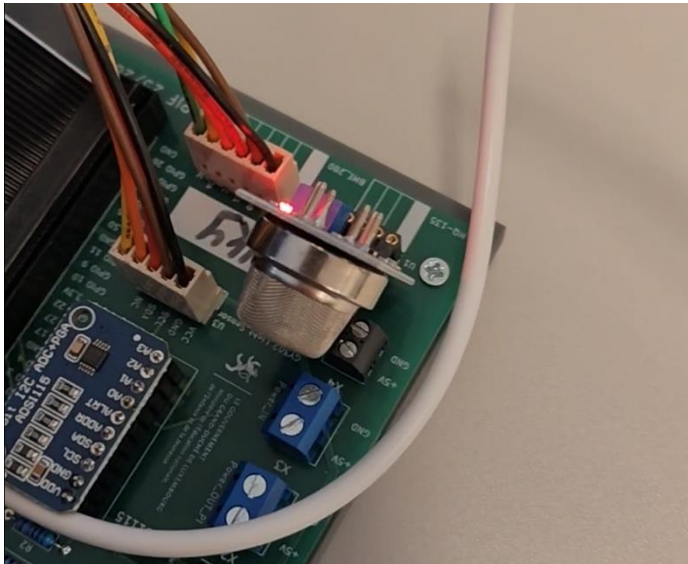
MQ135 Sensor Installation

For this assignment, no additional soldering was required.

The MQ135 gas sensor was inserted into its dedicated socket on the PCB.

The PCB is connected to the Raspberry Pi via GPIO pins and provides power and signal connections between the MQ135 sensor and the ADS1115 converter.

After installation, all components were visually inspected to make sure they were correctly placed and securely connected.



Sensor Testing

After powering on the Raspberry Pi, the I²C bus was scanned to check if the ADS1115 module was detected.

The following command was used:

```
sudo i2cdetect -y 1
```

The correct address appeared in the scan results, confirming that the converter was connected and working.

	0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
00:									--	--	--	--	--	--	--	--
10:	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--
20:	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--
30:	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--
40:	--	--	--	--	--	--	--	48	--	--	--	--	--	--	--	--
50:	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--
60:	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--
70:	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--

This address was then manually added to the Python script test-mq135.py, so the script could correctly communicate with the converter.

```
rudky718@rudky718: /opt/station/Firmware und Scripts
GNU nano 8.4 test-mq135.py
ADDRESS = 0x48 # 1000: Change this

from mq135 import MQ135
import smbus
def main():
    bus = smbus.SMBus(1) # Use 1 for Raspberry Pi, 0 for older models
    sensor = MQ135(bus, ADDRESS)

    try:
        gas = sensor.read_CO2()
        print(f"CO2 concentration: {gas} ppm")
    except Exception as e:
        print(f"Error reading CO2 concentration: {e}")

if __name__ == "__main__":
    main()
```

After updating the script, test-mq135.py was executed to read values from the MQ135 gas sensor. The script returned valid air quality data.

```
rudky718@rudky718:/opt/station/Firmware und Scripts $ python test-mq135.py
CO2 concentration: 4679 ppm
rudky718@rudky718:/opt/station/Firmware und Scripts $ python test-mq135.py
CO2 concentration: 4644 ppm
rudky718@rudky718:/opt/station/Firmware und Scripts $ python test-mq135.py
CO2 concentration: 3973 ppm
rudky718@rudky718:/opt/station/Firmware und Scripts $ python test-mq135.py
CO2 concentration: 3969 ppm
rudky718@rudky718:/opt/station/Firmware und Scripts $ python test-mq135.py
CO2 concentration: 4020 ppm
rudky718@rudky718:/opt/station/Firmware und Scripts $ python test-mq135.py
CO2 concentration: 3995 ppm
rudky718@rudky718:/opt/station/Firmware und Scripts $ python test-mq135.py
CO2 concentration: 3815 ppm
rudky718@rudky718:/opt/station/Firmware und Scripts $ python test-mq135.py
CO2 concentration: 3807 ppm
rudky718@rudky718:/opt/station/Firmware und Scripts $ python test-mq135.py
CO2 concentration: 3820 ppm
rudky718@rudky718:/opt/station/Firmware und Scripts $ python test-mq135.py
CO2 concentration: 4165 ppm
rudky718@rudky718:/opt/station/Firmware und Scripts $ python test-mq135.py
CO2 concentration: 4131 ppm
```

Conclusion

All connections and components were successfully tested and are working correctly.

The system can read data from the MQ135 gas sensor without errors.

However, the sensor still requires calibration, as the measured values do not fully match real air quality conditions.

The calibration can be done manually by adjusting the potentiometer on the sensor using a small screwdriver.

Sensor #2

List of materials

Component	Quantity
BME280 barometric (environmental) sensor	1
BH1750 light sensor	1
Plug-in cable	2

Introduction

The aim of this assignment is to connect and test the BH1750 light sensor and the BME280 environmental sensor.

These sensors are used to measure brightness, temperature, and air pressure. To allow Raspberry Pi to read the sensor data, the ADS1115 converter is used.

The task focuses on installing the sensors and verifying that the system works correctly.

Sensors Installation

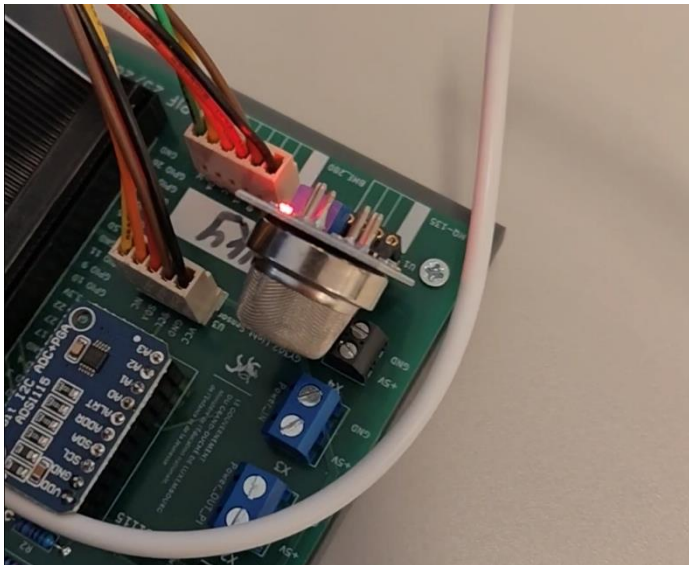
A plug-in cable was soldered to the BH1750 light sensor. Similar plug-in cable was also soldered to the BME280 environmental sensor.

During the installation of the BME280 module, two wires were not required. These unused wires were not connected to the sensor. To prevent short circuits, the open ends were covered with heat-shrink tubing. The tubing was placed over the wire ends and heated so that it shrank tightly around them. After heating, the ends were additionally pressed to fully close and protect the exposed wire.



All solder joints were visually inspected to ensure clean and stable connections. After checking the soldering quality, the sensors were ready to be connected.

The cables were then simply plugged into the circuit board. No additional soldering was required at this step.



Sensors Testing

After powering on the Raspberry Pi, the I²C bus was scanned to check if the modules were detected.

The following command was used:

```
sudo i2cdetect -y 1
```

The correct addresses appeared in the scan results, confirming that the modules were working.

```
rudky718@rudky718:/opt/station/Firmware und Scripts $ i2cdetect -y 1
    0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f
00:  -- -- -- -- -- -- -- -- -- -- -- -- -- --
10:  -- -- -- -- -- -- -- -- -- -- -- -- -- --
20:  -- -- -- 23 -- -- -- -- -- -- -- -- -- --
30:  -- -- -- -- -- -- -- -- -- -- -- -- -- --
40:  -- -- -- -- -- -- -- 48 -- -- -- -- -- --
50:  -- -- -- -- -- -- -- -- -- -- -- -- -- --
60:  -- -- -- -- -- -- -- -- -- -- -- -- -- --
70:  -- -- -- -- -- -- 76 -- -- -- -- -- --
```

These addresses were then manually added to the Python scripts test-bme280.py and test-bh1750.py, so the script could correctly communicate with the converter.


```
GNU nano 8.4 test-bme280.py

ADDRESS = 0x76 # 1000: Change this
from bme280 import BME280
import smbus

def main():
    bus = smbus.SMBus(1) # Use 1 for Raspberry Pi, 0 for older models
    sensor = BME280(bus, ADDRESS)
    try:
        temperature = sensor.read_temperature()
        pressure = sensor.read_pressure()
        humidity = sensor.read_humidity()

        print(f"Temperature: {temperature} °C")
        print(f"Pressure: {pressure} hPa")
        print(f"Humidity: {humidity} %")
    except Exception as e:
        print(f"Error reading BME280 sensor: {e}")

if __name__ == "__main__":
    main()
```

```
rudky718@rudky718: /opt/station/Firmware und Scripts
GNU nano 8.4 test-bh1750.py

ADDRESS = 0x23 # 1000: Change this

from bh1750 import BH1750
import smbus

def main():
    bus = smbus.SMBus(1) # Use 1 for Raspberry Pi, 0 for older models
    sensor = BH1750(bus, ADDRESS)

    try:
        light_level = sensor.read_light()
        print(f"Light Level: {light_level} lux")
    except Exception as e:
        print(f"Error reading light level: {e}")

if __name__ == "__main__":
    main()
```

After updating the scripts, they were executed to read values from the BH1750 light sensor and the BME280 environmental sensor. The light sensor script returned valid data. However, when running the test script for the BME280 sensor, an error appeared.

The problem was caused by the fact that the module was not a BME280, but a BMP280. Because of this, the original script was not compatible.

To solve the issue, the script was modified and the required library was installed.

```
python3 -m pip install bmp280 --break-system-packages
```

Script:

```
from bmp280 import BMP280
from smbus2 import SMBus
import time

ADDRESS = 0x76

def main():
    bus = SMBus(1)
    sensor = BMP280(i2c_dev=bus, i2c_addr=ADDRESS)

    while True:
        temperature = sensor.get_temperature()
        pressure = sensor.get_pressure()

        print(f"Temperature: {temperature:.2f} °C")
        print(f"Pressure: {pressure:.2f} hPa")
        print("-" * 30)
        time.sleep(2)

if __name__ == "__main__":
    main()
```

After these changes, the sensor worked correctly and returned valid measurement values.

Conclusion

Both sensors were successfully soldered, connected, and tested. The unused wires were safely insulated to avoid short circuits.

After configuring the I²C addresses and running the test programs, the sensors provided valid measurement values.

Web Database Server

Introduction

In this task a web server and a database server were deployed on an Ubuntu Server using Docker containers.

The goal of the project was to create a working environment where a website can connect to a database and be accessed through the network.

The system was implemented using the following components:

- Apache with PHP for the web server
- MariaDB for the database server
- phpMyAdmin for database administration
- Docker and Docker Compose for container management

All services run in separate containers but communicate through a Docker bridge network. The project also includes persistent database storage, Apache configuration, secure database configuration, and a test website to verify that the database connection works correctly.

Commands and Configuration

System Update

First the Ubuntu system was updated to make sure all packages were up to date

```
sudo apt update && sudo apt upgrade -y
```

Docker Installation

Docker and Docker Compose were installed together with required dependencies.

Install dependencies:

```
sudo apt install -y ca-certificates curl gnupg lsb-release
```

Add Docker GPG key:

```
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | \
sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
sudo chmod a+r /etc/apt/keyrings/docker.gpg
```

Add Docker repository:

```
echo \
"deb [arch=$(dpkg --print-architecture) \
signed-by=/etc/apt/keyrings/docker.gpg] \
https://download.docker.com/linux/ubuntu \
$(lsb_release -cs) stable" | \
sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

Install Docker Engine and Docker Compose:

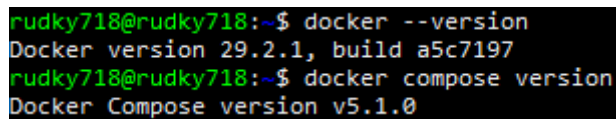
```
sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io \
docker-buildx-plugin docker-compose-plugin
```

Add the current user to the Docker group:

```
sudo usermod -aG docker $USER
newgrp docker
```

Verify Installation

```
docker --version
docker compose version
```



```
rudky718@rudky718:~$ docker --version
Docker version 29.2.1, build a5c7197
rudky718@rudky718:~$ docker compose version
Docker Compose version v5.1.0
```

Project Structure Creation

A project directory structure was created to organize the containers and configuration files.

```
mkdir -p ~/pif-server/{webserver,database,phpmyadmin,mysql-data,www}
cd ~/pif-server
```

Docker Compose Configuration

Docker Compose was used to define the three containers required for the system:

- MariaDB database server
- Apache + PHP web server
- phpMyAdmin interface

Configuration

version: '3.8'

services:

database:

```
image: mariadb:11
container_name: pif-database
restart: unless-stopped
environment:
  MYSQL_ROOT_PASSWORD: rootpassword
  MYSQL_DATABASE: pif_db
  MYSQL_USER: pif_user
  MYSQL_PASSWORD: pif_password
```

volumes:

- ./mysql-data:/var/lib/mysql

networks:

- pif-network

webserver:

build: ./webserver

container_name: pif-webserver

restart: unless-stopped

ports:

- "80:80"

volumes:

- ./www:/var/www/html

networks:

- pif-network

phpmyadmin:

image: phpmyadmin:latest

container_name: pif-phpmyadmin

restart: unless-stopped

ports:

- "8080:80"

environment:

PMA_HOST: database

networks:

- pif-network

networks:

pif-network:

driver: bridge

Webserver Docker file

The web server container was built using the official PHP Apache image.

FROM php:8.2-apache

RUN docker-php-ext-install mysqli pdo pdo_mysql

RUN a2enmod rewrite

COPY apache-config/000-default.conf /etc/apache2/sites-available/000-default.conf

File Permissions

Website files were configured with correct ownership and permissions.

```
sudo chown -R www-data:www-data ~/pif-server/www
```

```
sudo chmod -R 755 ~/pif-server/www
```

```
sudo chmod -R 644 ~/pif-server/www/*.php
```

```
rudky718@rudky718:~/pif-server$ ls -la ~/pif-server/www/
total 8
drwxr-xr-x 2 www-data www-data 4096 Mar  5 09:51 .
drwxrwxr-x 7 rudky718 docker  4096 Mar  5 10:03 ..
```

Start Docker Containers

The containers were started using Docker Compose.

```
docker compose up -d
```

Docker downloaded images, built the web server container and started the following services:

- pif-database
- pif-phpmyadmin

```
rudky718@rudky718:~/pif-server$ docker compose up -d
WARN[0000] /home/rudky718/pif-server/docker-compose.yml:
  remove it to avoid potential confusion
[+] up 34/34
  Image mariadb:11          Pulled
  Image phpmyadmin:latest   Pulled
[+] Building 43.7s (14/14) FINISHED
```

Secure MariaDB Installation

The MariaDB security script was executed inside the container.

```
docker exec -it pif-database bash
mysql_secure_installation
```

The script removed anonymous users, disabled remote root login, and removed the test database to secure the installation.

```

root@a3fbcecab33c:/# mariadb-secure-installation

NOTE: RUNNING ALL PARTS OF THIS SCRIPT IS RECOMMENDED FOR ALL MariaDB
SERVERS IN PRODUCTION USE! PLEASE READ EACH STEP CAREFULLY!

In order to log into MariaDB to secure it, we'll need the current
password for the root user. If you've just installed MariaDB, and
haven't set the root password yet, you should just press enter here.

Enter current password for root (enter for none):
OK, successfully used password, moving on...

Setting the root password or using the unix_socket ensures that nobody
can log into the MariaDB root user without the proper authorisation.

You already have your root account protected, so you can safely answer 'n'.

Switch to unix_socket authentication [Y/n] Y
Enabled successfully!
Reloading privilege tables..
... Success!

You already have your root account protected, so you can safely answer 'n'.

Change the root password? [Y/n] n
... skipping.

By default, a MariaDB installation has an anonymous user, allowing anyone
to log into MariaDB without having to have a user account created for
them. This is intended only for testing, and to make the installation
go a bit smoother. You should remove them before moving into a
production environment.

Remove anonymous users? [Y/n] Y
... Success!

Normally, root should only be allowed to connect from 'localhost'. This
ensures that someone cannot guess at the root password from the network.

Disallow root login remotely? [Y/n] Y
... Success!

By default, MariaDB comes with a database named 'test' that anyone can
access. This is also intended only for testing, and should be removed
before moving into a production environment.

Remove test database and access to it? [Y/n] Y
- Dropping test database...
... Success!
- Removing privileges on test database...
... Success!

Reloading the privilege tables will ensure that all changes made so far
will take effect immediately.

Reload privilege tables now? [Y/n] Y
... Success!

Cleaning up...

All done! If you've completed all of the above steps, your MariaDB
installation should now be secure.

Thanks for using MariaDB!

```


Database Configuration

A test database table and users were created.

SQL Commands

```
USE pif_db;
```

```
CREATE TABLE users (  
  username VARCHAR(50) PRIMARY KEY,  
  full_name VARCHAR(100),  
  email VARCHAR(100),  
  password_hash VARCHAR(200),  
  is_admin TINYINT(1)  
);
```

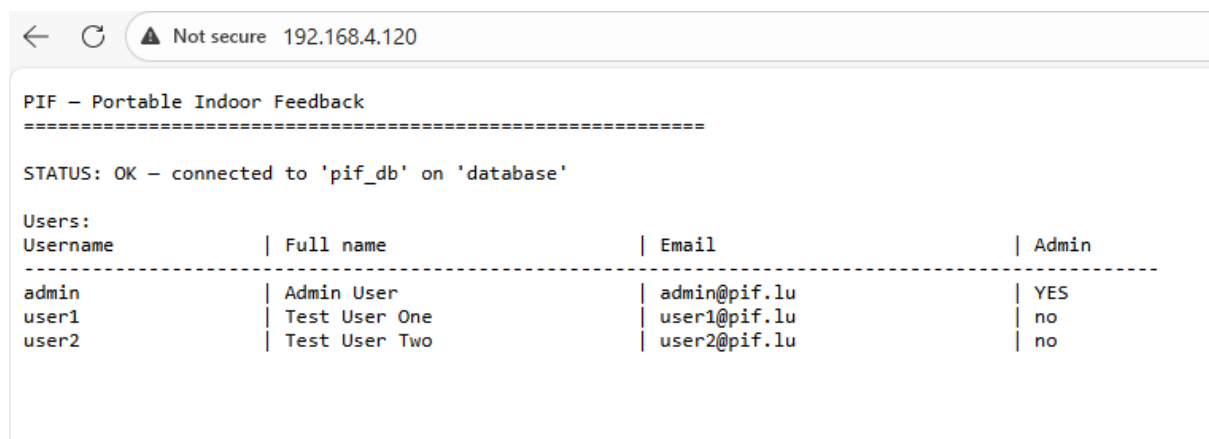
```
INSERT INTO users VALUES  
(  
  'admin','Admin User','admin@pif.lu',SHA2('admin123',256),1),  
(  
  'user1','Test User One','user1@pif.lu',SHA2('user123',256),0);
```

Verification

Website and Database Test

To verify that the web server and database were working correctly, a test PHP page was created in the www directory.

This page connects to the MariaDB database using the credentials defined in the Docker Compose configuration.



phpMyAdmin Test

To verify that the database could be accessed through phpMyAdmin, the following address was opened in the browser:

http://<server-ip>:8080



Welcome to phpMyAdmin

Language

English ▼

Log in

Username:

Password:

Log in

PHP Test

To verify that PHP was working correctly, a simple test file called test.php was created with the following code:

```
<?php  
phpinfo();
```

When the file was opened in the browser (<http://<server-ip>/test.php>), the PHP information page was displayed.

This confirmed that PHP was correctly installed and working with the Apache web server.

PHP Version 8.2.30



System	Linux be79da37df22 6.8.0-101-generic #101-Ubuntu SMP PREEMPT_DYNAMIC Mon Feb 9 10:15:05 UTC 2026 x86_64
Build Date	Feb 24 2026 19:13:57
Build System	Linux - Docker
Build Provider	https://github.com/docker-library/php
Configure Command	./configure '--build=x86_64-linux-gnu' '--sysconfdir=/usr/local/etc' '--with-config-file-path=/usr/local/etc/php' '--with-config-file-scan-dir=/usr/local/etc/php/conf.d' '--enable-option-checking=fatal' '--with-mhash' '--with-pic' '--enable-mbstring' '--enable-mysqlnd' '--with-password-argon2' '--with-sodium=shared' '--with-pdo-sqlite=/usr' '--with-sqlite3=/usr' '--with-curl' '--with-iconv' '--with-openssl' '--with-readline' '--with-zlib' '--disable-phpdbg' '--with-pear' '--with-libdir=lib/x86_64-linux-gnu' '--disable-cgi' '--with-apxs2' 'build_alias=x86_64-linux-gnu'
Server API	Apache 2.0 Handler
Virtual Directory Support	disabled
Configuration File (php.ini) Path	/usr/local/etc/php
Loaded Configuration File	/usr/local/etc/php/php.ini
Scan this dir for additional .ini files	/usr/local/etc/php/conf.d
Additional .ini files parsed	/usr/local/etc/php/conf.d/docker-php-ext-mysql.ini, /usr/local/etc/php/conf.d/docker-php-ext-openssl.ini, /usr/local/etc/php/conf.d/docker-php-ext-pdo_mysql.ini, /usr/local/etc/php/conf.d/docker-php-ext-sodium.ini
PHP API	20220829
PHP Extension	20220829
Zend Extension	420220829
Zend Extension Build	API420220829.NTS
PHP Extension Build	API20220829.NTS
Debug Build	no
Thread Safety	disabled
Zend Signal Handling	enabled
Zend Memory Manager	enabled
Zend Multibyte Support	provided by mbstring
Zend Max Execution Timers	disabled
IPv6 Support	enabled
DTrace Support	disabled
Registered PHP Streams	https, ftps, compress.zlib, php, file, glob, data, http, ftp, phar
Registered Stream Socket Transports	tcp, udp, unix, udg, ssl, tls, tlsv1.0, tlsv1.1, tlsv1.2, tlsv1.3
Registered Stream Filters	zlib.*, convert.iconv.*, string.rot13, string.toupper, string.tolower, convert.*, consumed, dechunk

This program makes use of the Zend Scripting Language Engine:
 Zend Engine v4.2.30, Copyright (c) Zend Technologies
 with Zend OPcache v8.2.30, Copyright (c), by Zend Technologies



Making sure all packages up to date

sudo apt update && sudo apt upgrade -y

cd ~/pif-server

docker compose pull

docker compose up -d

Conclusion

The project successfully deployed a complete web and database environment using Docker. Apache, PHP, MariaDB and phpMyAdmin were configured in separate containers connected through a Docker network.

The setup includes:

- Docker container orchestration using Docker Compose
- Apache web server with PHP support
- MariaDB database server with secure configuration
- phpMyAdmin for database administration
- Persistent database storage
- A test website to confirm successful communication between the web server and database server

This configuration provides a functional development environment that can be accessed through the network and easily maintained using Docker.

Assembly

List of materials

Component	Quantity
Wooden housing	1
Power button	1

Introduction

In this project, a wooden housing for a Raspberry Pi was assembled. The goal was to build a protective box using prepared wooden parts and to integrate additional hardware components such as a power button.

The project included assembling the wooden housing, preparing openings for hardware components, soldering the power button to the board, and installing two sensors inside the case.

Preparing the Opening for the Power Button

One of the tasks was to create an opening in the wooden panel for the power button. This button will later allow the Raspberry Pi to be turned on and off from outside the box.

First, the position of the hole was marked on the wooden panel. After that, a drill was used to create the opening. The hole was then checked to make sure the button fits correctly.

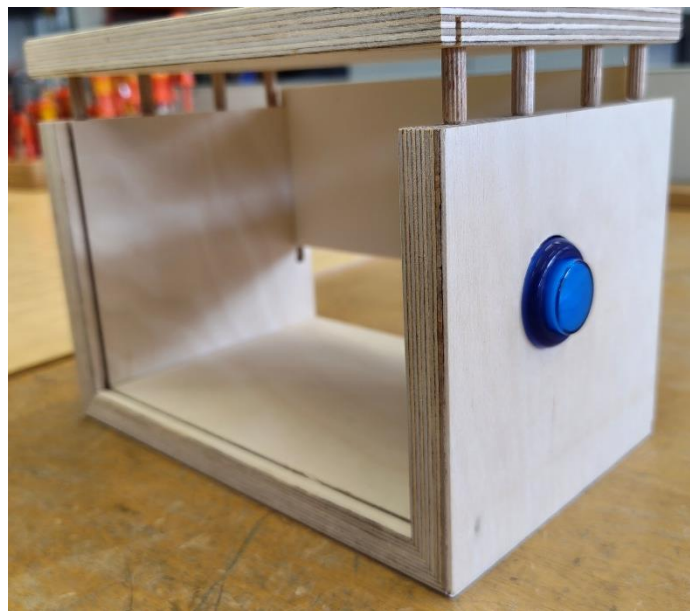
The edges of the hole were cleaned to make the opening smooth and ready for the button installation



Assembling the Wooden Box

After preparing the panel, the wooden box was assembled.

All wooden panels were connected using dowel pins. These pins fit into pre-made holes in the panels and help align the structure correctly.

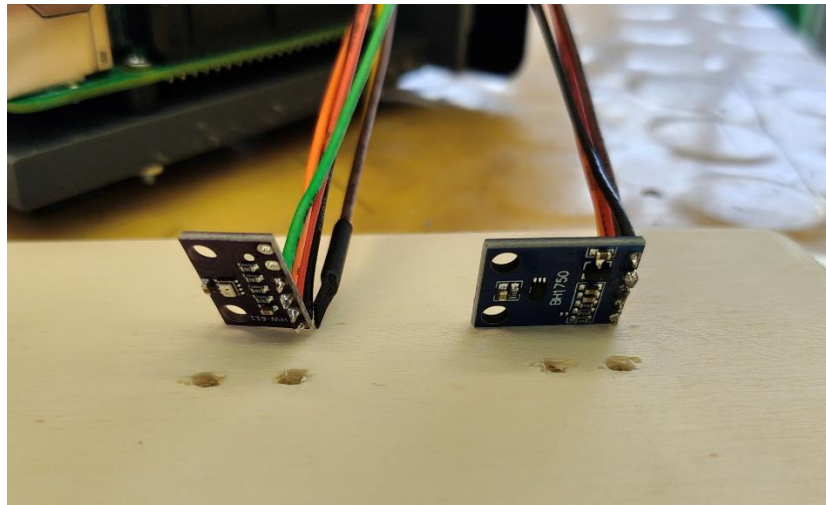


Mounting the Sensors

The next step was to install two sensors on the back panel of wooden housing. These sensors measure environmental data and send it to the Raspberry Pi.

Two openings were prepared in the wooden panel for the sensors. The sensors were inserted into these holes, so they remain stable and do not move.

The sensors were then secured using bolts and nuts so they stay firmly attached to the wooden panel.



Connecting the Power Button

The power button needed to be connected to the Raspberry Pi.

First, the wires were soldered to the contacts of the button. A rubber insulating sleeve was placed over the connection to protect it and prevent short circuits.

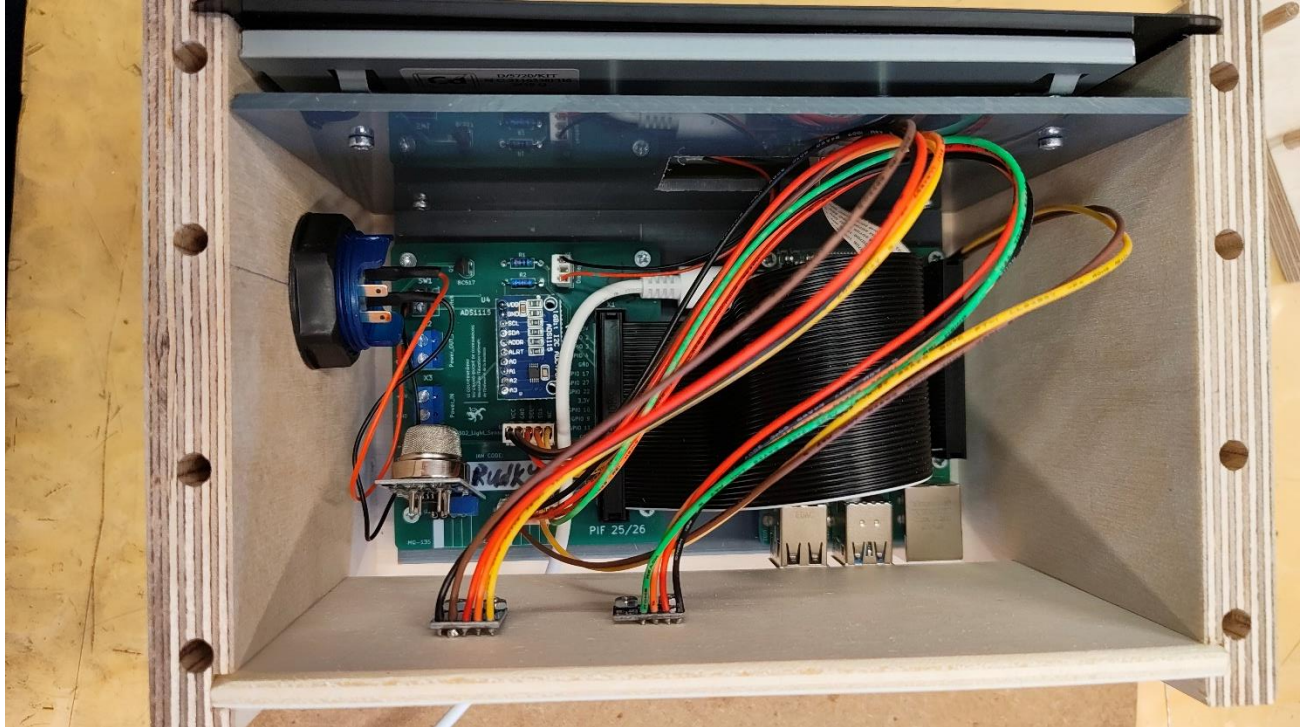
The other ends of the wires were connected to the Raspberry Pi board using screw terminals. The wires were inserted into the connectors and fixed in place by tightening the screws with a screwdriver.



Conclusion

During this sprint, the wooden enclosure for the Raspberry Pi was successfully assembled.

The wooden panels were connected to form a stable structure, and two sensors were installed on the panel. The power button was wired and connected to the Raspberry Pi using screw terminals.



Data Transfer

Introduction

In this task, a data transfer system was implemented for the PIF project.

The goal of this task is to receive measurement data from a Raspberry Pi station and store it in a MariaDB database using a PHP script. The system is based on the infrastructure created in the previous sprint (L4), which includes Docker containers for:

- Apache + PHP (web server)
- MariaDB (database server)
- phpMyAdmin (database management)

PHP Script Creation

First, the PHP script for receiving data was created.

```
nano ~/pif-server/www/data\_transfer.php
```

```

rudky718@rudky718:~$ ls -l ~/pif-server/www
total 12
-rwxr-xr-x 1 www-data www-data 1199 Mar 20 08:46 data_transfer.php
-rwxr-xr-x 1 www-data www-data 1640 Mar 11 07:31 index.php
-rwxr-xr-x 1 www-data www-data  17 Mar  5 14:55 test.php

```

This script performs the following actions:

- Connects to the database using PDO
- Receives data via HTTP POST
- Validates input data
- Maps station_serial to stationID
- Inserts data into the Measurements table

The script is accessible via:

http://<server-ip>/data_transfer.php

This is important because the Raspberry Pi sends data to this address.

Database Preparation

The database must contain a table for storing measurements.

```

MariaDB [pif_db]> show tables;
+-----+
| Tables_in_pif_db |
+-----+
| CollectionAccess  |
| CollectionMeasurement |
| Collections       |
| FriendList        |
| Measurements      |
| Stations          |
| Users             |
+-----+
7 rows in set (0.000 sec)

```

```
USE pif_db;
```

```

CREATE TABLE Measurements (
    measurementID INT AUTO_INCREMENT PRIMARY KEY,
    stationID INT NOT NULL,
    timestamp DATETIME NOT NULL,
    temperature FLOAT NOT NULL,
    humidity FLOAT NOT NULL,
    airPressure FLOAT NOT NULL,
    lightIntensity FLOAT NOT NULL,
    airQuality FLOAT NOT NULL,
    FOREIGN KEY(stationID) REFERENCES Stations(stationID) ON DELETE
    CASCADE
);

```

stationID connects measurements to a station

Station must exist:

```
INSERT INTO Stations (serialNumber, name)
VALUES ('1', 'Test Station');
```

```
MariaDB [pif_db]> SELECT * FROM Stations;
+-----+-----+-----+-----+-----+
| stationID | serialNumber | name       | description | ownerUserID |
+-----+-----+-----+-----+-----+
|          1 | 1           | Test Station | NULL        | NULL        |
+-----+-----+-----+-----+-----+
1 row in set (0.000 sec)
```

This is required because the PHP script maps station_serial to stationID.

Testing Data Transfer

To verify that the PHP script works, a manual request was sent using curl:

```
curl -X POST http://192.168.4.15/data_transfer.php \
-d "station_serial=1" \
-d "timestamp=2026-03-20 14:50:00" \
-d "temperature=25" \
-d "humidity=50" \
-d "pressure=1000" \
-d "light=10" \
-d "gas=500"
```

Expected result: OK

```
rudky718@rudky718:~$ curl -X POST http://localhost/data_transfer.php \
> -d "stationID=1" \
> -d "timestamp=2026-03-19 15:30:00" \
> -d "temperature=22.5" \
> -d "humidity=45.0" \
> -d "airPressure=1012.0" \
> -d "lightIntensity=300.0" \
> -d "airQuality=50.0"
OKrudky718@rudky718:~$
```

Result in Measurements table:

```
MariaDB [pif_db]> SELECT * FROM Measurements;
+-----+-----+-----+-----+-----+-----+-----+
| measurementID | stationID | timestamp           | temperature | humidity | airPressure | lightIntensity | airQuality |
+-----+-----+-----+-----+-----+-----+-----+
|              2 |          1 | 2026-03-19 15:30:00 | 22.5        | 45        | 1012        | 300           | 50         |
+-----+-----+-----+-----+-----+-----+-----+
1 row in set (0.001 sec)
```

This confirms that the script successfully inserts data into the database.

Raspberry Pi Configuration

The Raspberry Pi uses a configuration file:

```
nano /opt/station/config.toml
```

```

GNU nano 8.4 /opt/station/config.toml
station_serial = 1          # Serial number of the station

interval = 2                # Time between measurements

toggle_intensity = 100      # The light level at which the LED should be turned on

[i2c]                       # I2C-Addresses of the sensors
bme280 = 0x76
bh1750 = 0x23
mq135 = 0x48

[destination]               # Information about the server
ip = "192.168.4.15"         # IP-Address or Hostname of the Server
path = "/data_transfer.php" # Path to the script that accepts the measurements

```

Important:

- **ip** is the server address
- **path** is the location of the PHP script
- **station_serial** is used to link the data in the database

Verification

To confirm that everything works:

Check database:

```
docker exec -it pif-database mariadb -u root -p pif_db \
```

```
-e "SELECT * FROM Measurements ORDER BY measurementID DESC LIMIT 5;"
```

Or check phpMyAdmin

measurementID	stationID	timestamp	temperature	humidity	airPressure	lightIntensity	airQuality
2	1	2026-03-19 15:30:00	22.5	45	1012	300	50
3	1	2026-03-19 15:30:00	22.5	45	1012	300	50
4	1	2026-03-19 15:30:00	22.5	45	1012	300	50
5	1	2026-03-20 14:50:00	25	50	1000	10	500
6	1	2026-03-20 14:50:00	25	50	1000	10	500
7	1	2026-03-20 14:24:44	22.8418	0	639.024	0.833333	3657
8	1	2026-03-20 14:24:46	24.9443	0	989.426	0.833333	3655
9	1	2026-03-20 14:24:48	24.9453	0	989.422	0.833333	3659
10	1	2026-03-20 14:24:50	24.9447	0	989.433	0.833333	3659
11	1	2026-03-20 14:24:52	24.9408	0	989.42	0.833333	3659
12	1	2026-03-20 14:24:54	24.9364	0	989.414	0.833333	3657

Data transfer is working correctly if new rows appear every few seconds

Manual Data Entry Form

To improve usability, a manual input form was added to the PHP script.

The form allows users to:

- Enter station serial number
- Enter measurement values
- Submit data to the database

The form is accessible:

http://<server_ip>/data_transfer.php

Explanation:

- step="0.01" allows decimal input (e.g. 25.17)
- Validation prevents incorrect data

Result in Measurements table:

478	1	2026-03-20 14:59:34	24.0375	0	989.023	0.833333	3492
479	1	2026-03-20 14:59:36	24.0334	0	989.021	0.833333	3491
480	1	2026-03-20 14:57:00	24	50	900	1000	3500
481	1	2026-03-20 14:59:38	24.0299	0	989.024	0.833333	3489
482	1	2026-03-20 14:59:40	24.0324	0	989.018	0.833333	3487

Conclusion

The data transfer system was successfully implemented. The Raspberry Pi sends measurement data via HTTP POST to the PHP script, which stores it in the MariaDB database. All components (Docker, PHP, MariaDB, Raspberry Pi) work together as a complete system.

Code

```
<?php
$host = 'database';
$db   = 'pif_db';
$user = 'pif_user';
$pass = 'pif_password';

try {
    $pdo = new PDO("mysql:host=$host;dbname=$db;charset=utf8", $user, $pass);
    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
} catch (PDOException $e) {
    die("DB ERROR: " . $e->getMessage());
}

$message = "";

// Handle POST (form or measure.py)
if ($_SERVER['REQUEST_METHOD'] === 'POST') {

    $station_serial = $_POST['station_serial'] ?? null;
    $timestamp      = $_POST['timestamp'] ?? null;
    $temperature    = $_POST['temperature'] ?? null;
    $humidity       = $_POST['humidity'] ?? null;
    $airPressure    = $_POST['pressure'] ?? null;
    $lightIntensity = $_POST['light'] ?? null;
    $airQuality     = $_POST['gas'] ?? null;

    // Validation
    if (!$station_serial || !$timestamp) {
        $message = "ERROR: station_serial and timestamp required";
    } elseif (!is_numeric($temperature) || !is_numeric($humidity)) {
        $message = "ERROR: Invalid numeric values";
    } else {

        // Get stationID
        $stmt = $pdo->prepare("SELECT stationID FROM Stations WHERE
serialNumber=?");
        $stmt->execute([$station_serial]);
        $stationID = $stmt->fetchColumn();

        if (!$stationID) {
            $message = "ERROR: Unknown station serial";
        }
    }
}
```

```

    } else {

        // Insert
        $stmt = $pdo->prepare("
            INSERT INTO Measurements
            (stationID, timestamp, temperature, humidity, airPressure,
lightIntensity, airQuality)
            VALUES (?, ?, ?, ?, ?, ?, ?)
        ");

        $stmt->execute([
            $stationID,
            $timestamp,
            $temperature,
            $humidity,
            $airPressure,
            $lightIntensity,
            $airQuality
        ]);

        $message = "Data inserted successfully!";
    }
}
?>

<!DOCTYPE html>
<html>
<head>
    <title>Manual Data Entry</title>
    <style>
        body {
            font-family: Arial;
            background: #f4f4f4;
            text-align: center;
        }
        form {
            background: white;
            padding: 20px;
            margin: 40px auto;
            width: 300px;
            border-radius: 10px;
            box-shadow: 0 0 10px gray;
        }
        input {
            width: 90%;
            padding: 8px;
            margin: 5px;
        }
        button {
            padding: 10px;

```

```

        background: #4CAF50;
        color: white;
        border: none;
        cursor: pointer;
    }
    .msg {
        margin-top: 10px;
        font-weight: bold;
    }
</style>
</head>
<body>

<h2>Manual Measurement Entry</h2>

<form method="POST">
    <input type="text" name="station_serial" placeholder="Station Serial"
required><br>
    <input type="datetime-local" name="timestamp" required><br>
    <input type="number" step="0.01" name="temperature"
placeholder="Temperature"><br>
    <input type="number" step="0.01" name="humidity" placeholder="Humidity"><br>
    <input type="number" step="0.01" name="pressure" placeholder="Pressure"><br>
    <input type="number" step="0.01" name="light" placeholder="Light"><br>
    <input type="number" step="0.01" name="gas" placeholder="Air Quality"><br>

    <button type="submit">Submit</button>
</form>

<div class="msg"><?php echo $message; ?></div>

</body>
</html>

```

Lighting

List of materials

Component	Quantity
LED strip	1
Cable	1

Introduction

An opening was created on the side of the station near the screw terminal labeled LED_strip on the board.

This terminal is used to connect and fix wires by tightening screws. The hole allows the wire to pass from outside into the station and reach this terminal.

Preparing the Opening

An opening was created on the side of the station near the screw terminal on the board.

This terminal is used to connect and fix wires by tightening screws. The hole allows the wire to pass from outside into the station and reach the board.

The position of the hole was chosen carefully to ensure easy access to the terminal.



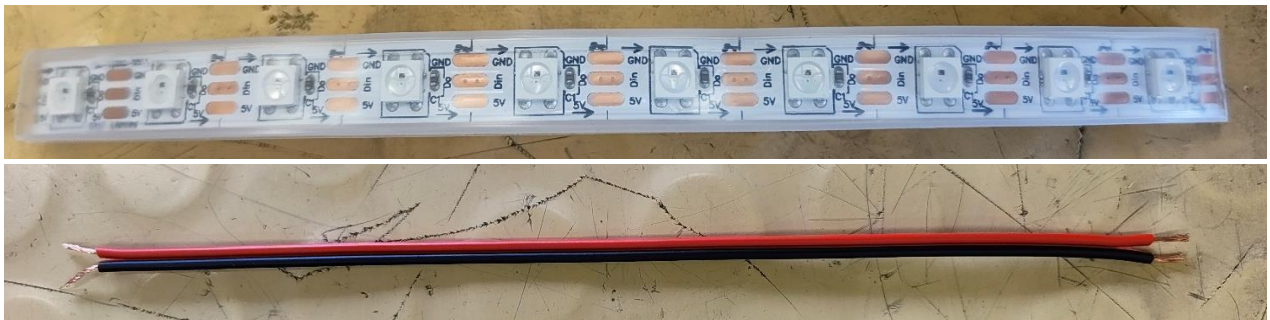
Preparing the LED Strip and Wires

The LED strip was cut to the required length. It is placed on the front side of the station.

After that, wires of the correct length were cut. The wires must be long enough to reach from the LED strip to the board inside the station.

The ends of the wires were stripped to prepare them for connection.

The wire enters from the side through the prepared opening and connects to the board inside.



Connecting Components

One end of the wire was connected to the screw terminal on the board and fixed by tightening the screw.

The other end of the wire was soldered to the LED strip.

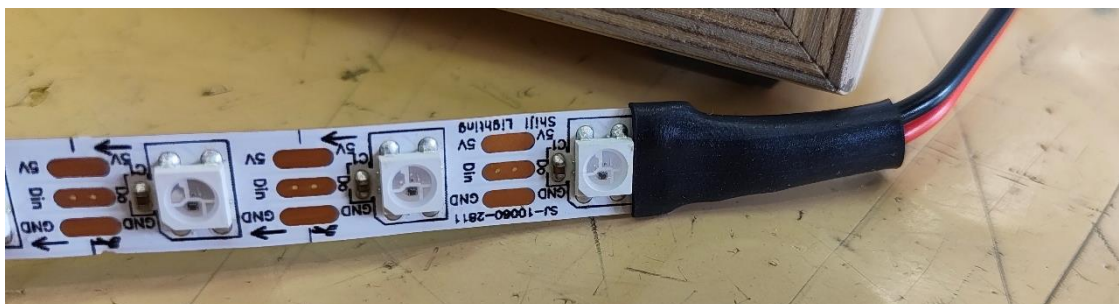
It is important that the same wire is connected to ground (GND) on both the Raspberry Pi board and the LED strip. Otherwise, the LED strip will not work.



Insulation and Mounting

After soldering, heat shrink tubing was applied to the connections to ensure proper insulation and protection.

Finally, the LED strip was attached to the front side of the station using hot-melt adhesive.



Conclusion

During this sprint, the LED strip was successfully installed and connected to the Raspberry Pi station. All physical steps, including cutting, wiring, soldering, and insulation, were completed.

The LED strip is designed to turn on when the brightness value is lower than 100. However, at the current stage, the LED strip does not work as expected.



Backup Server

Introduction

The purpose of this assignment is to extend the existing system by implementing a backup server for the database server.

A separate backup server is used to automatically create database backups and store them safely. This improves reliability and ensures that data can be restored in case of system failure.

Web/DB Server Configuration

Allow External Access to MariaDB

The database runs inside Docker and is not accessible externally by default. To allow the backup server to connect, port 3306 must be exposed.

Open the configuration file:

```
nano ~/pif-server/docker-compose.yml
```

Add port mapping to the database service:

ports:

- "3306:3306"

```

GNU nano 7.2 /home/rudky718/pif-server/docker-compose.yml
version: '3.8'

services:
  # — Контейнер 1: MariaDB (сервер баз даних) —————
  database:
    image: mariadb:11
    container_name: pif-database
    restart: unless-stopped
    ports:
      - "3306:3306"
    environment:
      MYSQL_ROOT_PASSWORD: rootpassword      # Змінити на власний пароль!
      MYSQL_DATABASE: pif_db
      MYSQL_USER: pif_user
      MYSQL_PASSWORD: pif_password          # Змінити на власний пароль!
    volumes:
      - ./mysql-data:/var/lib/mysql          # Постійне зберігання даних
    networks:
      - pif-network
    healthcheck:
      test: ["CMD", "healthcheck.sh", "--connect", "--innodb_initialized"]
      interval: 10s
      timeout: 5s
      retries: 5

```

Restart containers:

```

cd ~/pif-server
docker compose down
docker compose up -d

```

Verify that the port is open:

```
docker compose ps
```

The output should contain:

```
0.0.0.0:3306->3306/tcp
```

```

rudky718@rudky718:~/pif-server$ docker compose ps
WARN[0000] /home/rudky718/pif-server/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please
remove it to avoid potential confusion

```

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
pif-database	mariadb:11	"docker-entrypoint.s..."	database	5 hours ago	Up 58 minutes (healthy)	0.0.0.0:3306->3306/tcp, [::]:3306->3306/tcp
pif-phpmyadmin	phpmyadmin:latest	"/docker-entrypoint..."	phpmyadmin	5 hours ago	Up 58 minutes	0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
pif-webserver	pif-server-webserver	"docker-php-entrypoi..."	webserver	5 hours ago	Up 58 minutes	0.0.0.0:80->80/tcp, [::]:80->80/tcp, 0.0.0.0:443->443/tcp, [::]:443->443/tcp

Create Backup User

A separate database user is created for backup operations.
This improves security by avoiding the use of the root account.

Connect to MariaDB:

```
docker exec -it pif-database mariadb -u root -p
```

Run SQL commands:

```
CREATE USER 'backup_user'@'%' IDENTIFIED BY 'backup_password';
```

```
GRANT SELECT, LOCK TABLES, SHOW VIEW, EVENT, TRIGGER  
ON pif_db.*  
TO 'backup_user'@'%';
```

```
FLUSH PRIVILEGES;
```

This user has only the required permissions for backup.

```
MariaDB [pif_db]> SELECT User, Host FROM mysql.user;  
+-----+-----+  
| User      | Host      |  
+-----+-----+  
| backup_user | %         |  
| pif_user   | %         |
```

Backup Server Setup

System configuration

Update system packages:

```
sudo apt update && sudo apt upgrade -y
```

Set timezone:

```
sudo timedatectl set-timezone Europe/Luxembourg  
timedatectl
```

Install client to connect to the remote database:

```
sudo apt install -y mariadb-client
```

Backup Directory

Create directory for storing backups:

```
sudo mkdir -p /backup/mysql  
sudo chown rudky718:rudky718 /backup  
sudo chmod 755 /backup  
sudo chown rudky718:rudky718 /backup/mysql  
sudo chmod 700 /backup/mysql
```

The /backup/mysql directory is restricted to the owner for security.

Backup Script

Create script file:

nano /backup/backup_mysql.sh

 rudky718@backupserverrudky718: ~

```
GNU nano 7.2 /backup/backup_mysql.sh
#!/bin/bash

# conn
DB_NAME="pif_db"
DB_HOST="192.168.4.97"      # IP DB Server
DB_USER="backup_user"
DB_PASS="backup_password"

BACKUP_DIR="/backup/mysql"
DATE=$(date +%Y-%m-%d_%H-%M-%S)
BACKUP_FILE="$BACKUP_DIR/${DB_NAME}_${DATE}.sql"
LOG_FILE="/backup/backup.log"
RETENTION_DAYS=7

# Logs
log() {
    echo "[$(date +%Y-%m-%d %H:%M:%S)] $1" >> "$LOG_FILE"
}

log "INFO: Starting backup of $DB_NAME"

# Backup
mariadb-dump -h "$DB_HOST" \
    -u "$DB_USER" \
    -p"$DB_PASS" \
    --single-transaction \
    --routines \
    --triggers \
    "$DB_NAME" > "$BACKUP_FILE" 2>> "$LOG_FILE"

# Check
if [ $? -eq 0 ]; then
    log "SUCCESS: Backup saved to $BACKUP_FILE"
else
    log "ERROR: Backup failed!"
    exit 1
fi

# Old files deleting
find "$BACKUP_DIR" -name "*.sql" \
    -mtime +$RETENTION_DAYS -delete

log "INFO: Old backups cleaned up"
```

Make Script Executable and Test

chmod +x /backup/backup_mysql.sh

bash /backup/backup_mysql.sh

Verify backup:

ls -la /backup/mysql/

cat /backup/backup.log

```

rudky718@backupserverrudky718:~$ ls -la /backup/mysql/
total 1336
drwxr-xr-x 2 ftpuser ftpuser 4096 Apr 17 13:17 .
drwxr-xr-x 3 rudky718 rudky718 4096 Apr 18 22:30 ..
-rw-rw-r-- 1 ftpuser ftpuser 1804 Apr 17 09:33 pif_db_2026-04-17_09-33-01.sql
-rw-rw-r-- 1 ftpuser ftpuser 1804 Apr 17 09:47 pif_db_2026-04-17_09-47-36.sql
-rw-rw-r-- 1 ftpuser ftpuser 1804 Apr 17 10:17 pif_db_2026-04-17_10-17-49.sql
-rw-rw-r-- 1 ftpuser ftpuser 1804 Apr 17 10:24 pif_db_2026-04-17_10-24-38.sql
-rw-rw-r-- 1 ftpuser ftpuser 438367 Apr 17 10:36 pif_db_2026-04-17_10-36-17.sql
-rw-rw-r-- 1 ftpuser ftpuser 0 Apr 17 13:15 pif_db_2026-04-17_13-15-22.sql
-rw-rw-r-- 1 ftpuser ftpuser 449101 Apr 17 13:16 pif_db_2026-04-17_13-16-17.sql
-rw-rw-r-- 1 ftpuser ftpuser 450482 Apr 17 13:17 pif_db_2026-04-17_13-17-02.sql

```

Cron Automation

Open crontab:

```
crontab -e
```

Add:

```
0 8 * * * /backup/backup_mysql.sh
```

Check cron jobs:

```
crontab -l
```

Check service:

```
sudo systemctl status cron
```

```
sudo systemctl enable cron
```

This runs the backup every day at 08:00.

```

# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h dom mon dow   command
# Backup everyday at 8:00
0 8 * * * /backup/backup_mysql.sh

```

FTP

Install FTP Server

```
sudo apt install -y vsftpd
```

Configure FTP

Backup configuration file:

```
sudo cp /etc/vsftpd.conf /etc/vsftpd.conf.bak
```

Edit configuration:

```
sudo nano /etc/vsftpd.conf
```

Set parameters:

```
anonymous_enable=NO
local_enable=YES
write_enable=YES
chroot_local_user=YES
allow_writeable_chroot=YES
local_root=/backup/mysql
pasv_enable=YES
pasv_min_port=40000
pasv_max_port=50000
```

Create FTP User

```
sudo adduser ftpuser
```

Set permissions:

```
sudo chown -R ftpuser:ftpuser /backup/mysql
sudo chmod 755 /backup/mysql
```

Restart service:

```
sudo systemctl restart vsftpd
sudo systemctl enable vsftpd
sudo systemctl status vsftpd
```

Test FTP Connection

```
ftp 192.168.X.Y
```

Login:

Username: ftpuser

Password: ftpuser

Check files:

```
ls
```



```

rudky718@backupserverrudky718:~$ sudo adduser ftpuser
info: Adding user `ftpuser' ...
info: Selecting UID/GID from range 1000 to 59999 ...
info: Adding new group `ftpuser' (1001) ...
info: Adding new user `ftpuser' (1001) with group `ftpuser (1001)' ...
warn: The home directory `/home/ftpuser' already exists. Not touching this directory.
New password:
Retype new password:
passwd: password updated successfully
Changing the user information for ftpuser
Enter the new value, or press ENTER for the default
    Full Name []:
    Room Number []:
    Work Phone []:
    Home Phone []:
    Other []:
Is the information correct? [Y/n]
info: Adding new user `ftpuser' to supplemental / extra groups `users' ...
info: Adding user `ftpuser' to group `users' ...
rudky718@backupserverrudky718:~$ sudo chown -R ftpuser:ftpuser /backup/mysql
rudky718@backupserverrudky718:~$ sudo chmod 755 /backup/mysql
rudky718@backupserverrudky718:~$ sudo systemctl restart vsftpd
rudky718@backupserverrudky718:~$ ftp 192.168.4.116
Connected to 192.168.4.116.
220 (vsFTPd 3.0.5)
Name (192.168.4.116:rudky718): ftpuser
331 Please specify the password.
Password:
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp>

```

Conclusion

In this assignment, a backup system was successfully implemented using a separate server.

The database server was configured to allow external access, and a dedicated backup user was created with limited permissions.

The backup server automatically creates daily backups using a shell script and cron.

FTP access was configured to allow easy download of backup files from another machine.

Code

```

#!/bin/bash
# conn
DB_NAME="pif_db"
DB_HOST="192.168.4.97"          # IP DB Server
DB_USER="backup_user"
DB_PASS="backup_password"
BACKUP_DIR="/backup/mysql"
DATE=$(date +%Y-%m-%d_%H-%M-%S)
BACKUP_FILE="$BACKUP_DIR/${DB_NAME}_${DATE}.sql"
LOG_FILE="/backup/backup.log"
RETENTION_DAYS=7
# Logs
log() {
    echo "[$(date +%Y-%m-%d %H:%M:%S)] $1" >> "$LOG_FILE"
}

```

```

}
log "INFO: Starting backup of $DB_NAME"
# Backup
mariadb-dump -h "$DB_HOST" \
  -u "$DB_USER" \
  -p"$DB_PASS" \
  --single-transaction \
  --routines \
  --triggers \
  "$DB_NAME" > "$BACKUP_FILE" 2>> "$LOG_FILE"
# Check
if [ $? -eq 0 ]; then
  log "SUCCESS: Backup saved to $BACKUP_FILE"
else
  log "ERROR: Backup failed!"
  exit 1
fi

# Old files deleting
find "$BACKUP_DIR" -name "*.sql" \
  -mtime +$RETENTION_DAYS -delete

log "INFO: Old backups cleaned up"

```